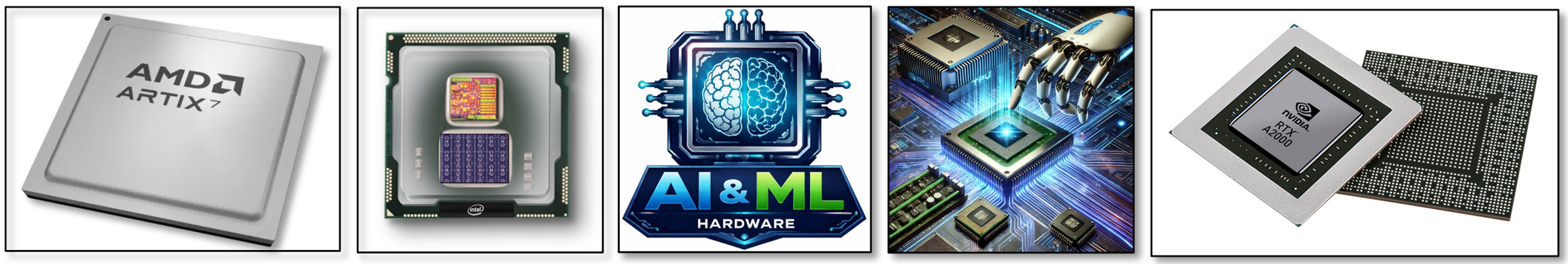
Christof Teuscher

ECE 410/510: Hardware for AI and ML, 2026

Transformers + In-memory computing

Portland State University
Department of Electrical and Computer Engineering (ECE)
www.teuscher-lab.com
teuscher@pdx.edu

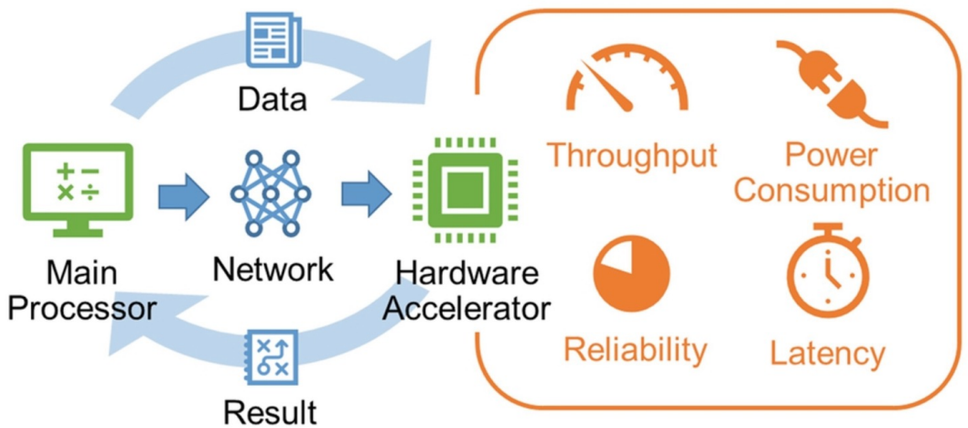


teuscher Lab
teuscher-lab.com

Portland State
UNIVERSITY

How can we accelerate an algorithm?

Or in other words: solve a problem as fast as possible?



How can we accelerate an algorithm?

- Technology scaling (but Moore's law is flattening out)
- Cache optimizations
- Code optimization
- Exploiting parallelism
- GPUs
- TPUs
- Fancier CPUs? More pipelining? Faster storage? Better networking?
- HW/SW co-design
- Improve the algorithm (trade time for space, approximations, etc.)
- Emerging technology

Traditional technology

Optimization Technique	Potential Speedup	Difficulty	Applicability	Notes
Algorithm Selection				Changing from $O(n^2)$ to $O(n \log n)$ yields massive gains at scale
Data Structure Optimization				E.g., Hash tables vs. arrays for lookups
GPU Acceleration				For parallelizable computations only
Multithreading/Parallelization				Limited by number of cores and parallelizable portions
Compiler Optimizations				Simply changing compiler flags
Memory Hierarchy Optimization				Cache-friendly data structures and access patterns
Memoization/Caching				For algorithms with repeated calculations
Hardware Upgrade (CPU)				Generational improvement in processors
SSD vs HDD Storage				For I/O-bound algorithms
FPGA/ASIC Implementation				Custom hardware, high development cost
Code Micro-optimizations				Loop unrolling, reducing function calls, etc.
Language Change (Interpreted to Compiled)				E.g., Python to C++

Traditional technology

Optimization Technique	Potential Speedup	Difficulty	Applicability	Notes
Algorithm Selection	10x-1000x+	Medium-High	High	Changing from $O(n^2)$ to $O(n \log n)$ yields massive gains at scale
Data Structure Optimization	2x-100x	Medium	High	E.g., Hash tables vs. arrays for lookups
GPU Acceleration	10x-1000x	High	Medium	For parallelizable computations only
Multithreading/Parallelization	2x-64x	High	Medium	Limited by number of cores and parallelizable portions
Compiler Optimizations	1.2x-4x	Low	High	Simply changing compiler flags
Memory Hierarchy Optimization	2x-10x	Medium	High	Cache-friendly data structures and access patterns
Memoization/Caching	2x-100x	Low-Medium	Medium	For algorithms with repeated calculations
Hardware Upgrade (CPU)	1.5x-4x	Low	High	Generational improvement in processors
SSD vs HDD Storage	2x-100x	Low	Medium	For I/O-bound algorithms
FPGA/ASIC Implementation	10x-1000x	Very High	Low	Custom hardware, high development cost
Code Micro-optimizations	1.1x-2x	Medium	High	Loop unrolling, reducing function calls, etc.
Language Change (Interpreted to Compiled)	3x-50x	High	Medium	E.g., Python to C++

Christof Teuscher teuscher@pdx.edu

Emerging technology

Technology	Potential Speedup	Energy Efficiency Gain	Technology Readiness	Key Advantages	Major Challenges
Memristors	10x-100x	100x-1000x	Medium-High	In-memory computing, storage/compute unification, analog synaptic behavior	Device variability, endurance, switching uniformity
Neuromorphic Chips	10x-1000x	1000x-10,000x	Medium	Brain-inspired architectures, spike-based computing, massive parallelism	Programming paradigm, algorithms development, scaling to complex tasks
Quantum Computing	Exponential for specific problems	Varies widely	Low	Quantum parallelism, exponential speedup for specialized problems	Decoherence, error correction, limited algorithm scope, cryogenic requirements
Spintronics	5x-50x	10x-100x	Medium	Non-volatile, minimal heat generation, high endurance	Integration with conventional CMOS, scalability issues
Photonic Computing	100x-1000x	100x-1000x	Low-Medium	Ultra-high bandwidth, minimal heat, wave-based parallel processing	Integration challenges, photonic-electronic interfaces
Memcapacitors	10x-100x	50x-500x	Low	Complementary to memristors, charge-based storage, improved switching	Early research stage, fabrication challenges
Reservoir Computing	50x-500x for temporal tasks	100x-1000x	Low-Medium	Efficient time-series processing, reduced training complexity	Limited to specific task domains, generalization issues
DNA Computing	Massive parallelism (theoretical)	Ultra-low (theoretical)	Very Low	Enormous parallelism, molecular-scale computing, energy efficiency	Speed limitations, error rates, interface challenges
Phase-Change Materials	5x-50x	10x-100x	Medium-High	Multi-level storage, established manufacturing, non-volatile	Energy consumption for phase transition, endurance
Protein-based Nanowires	Unknown	10,000x-100,000x (theoretical)	Very Low	Ultra-low power consumption, biocompatibility	Early research stage, manufacturing scalability

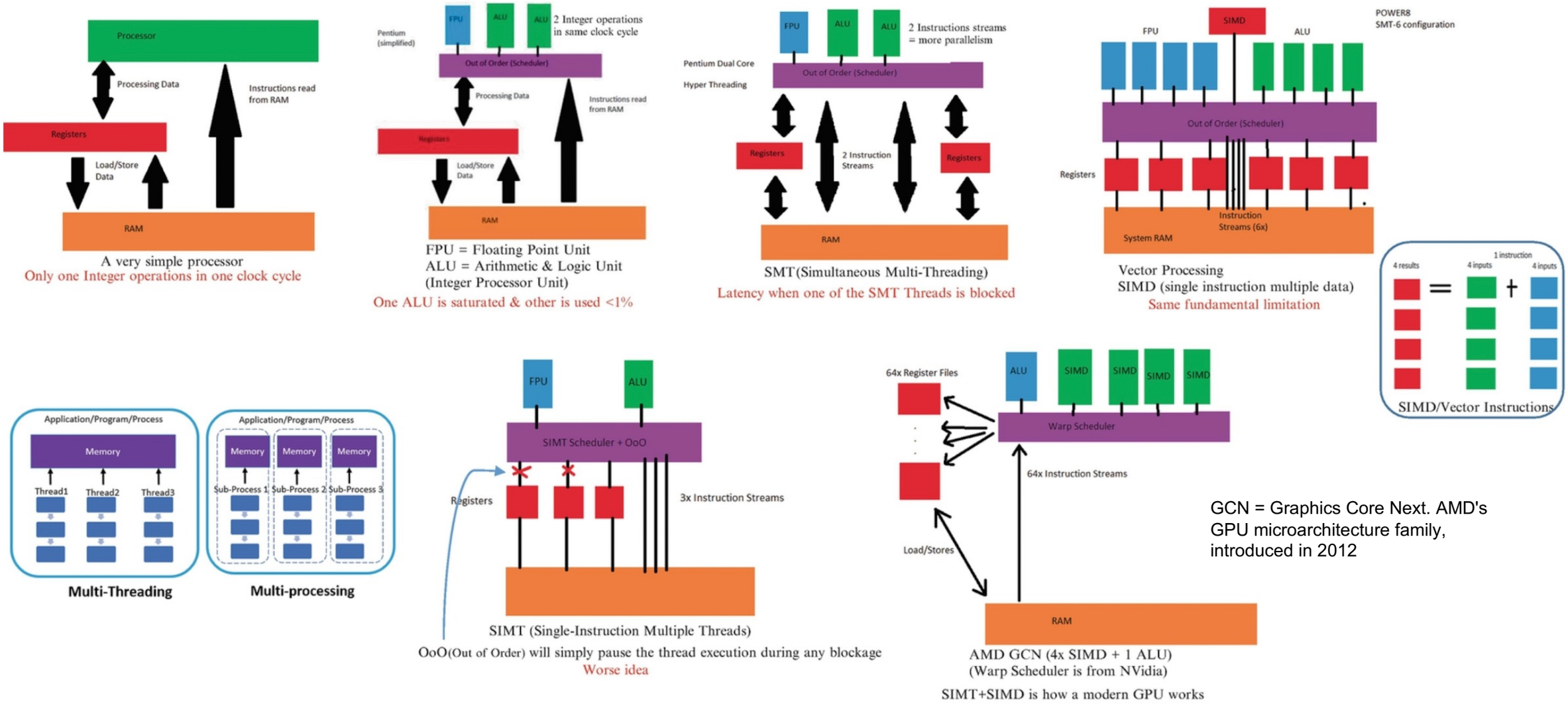
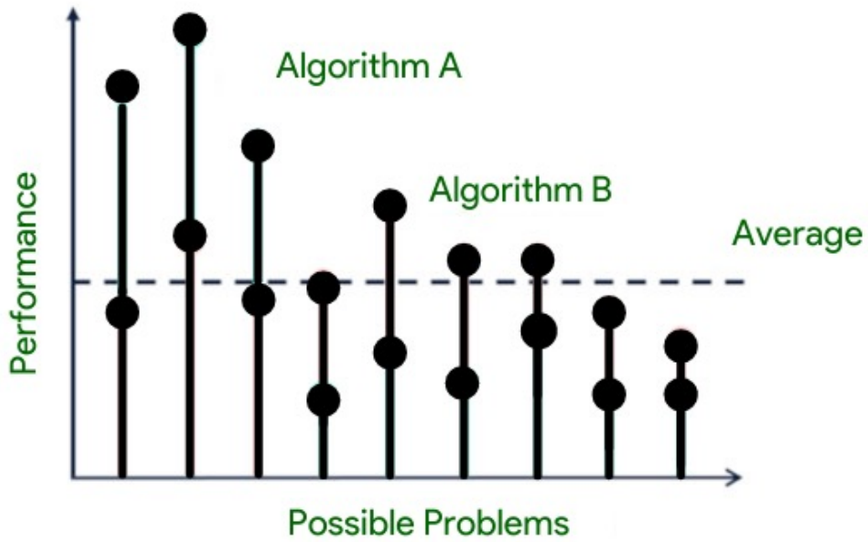


Fig. 14 A few examples of processor designs used for hardware acceleration

No Free Lunch theorem (NFL)

QUIZ

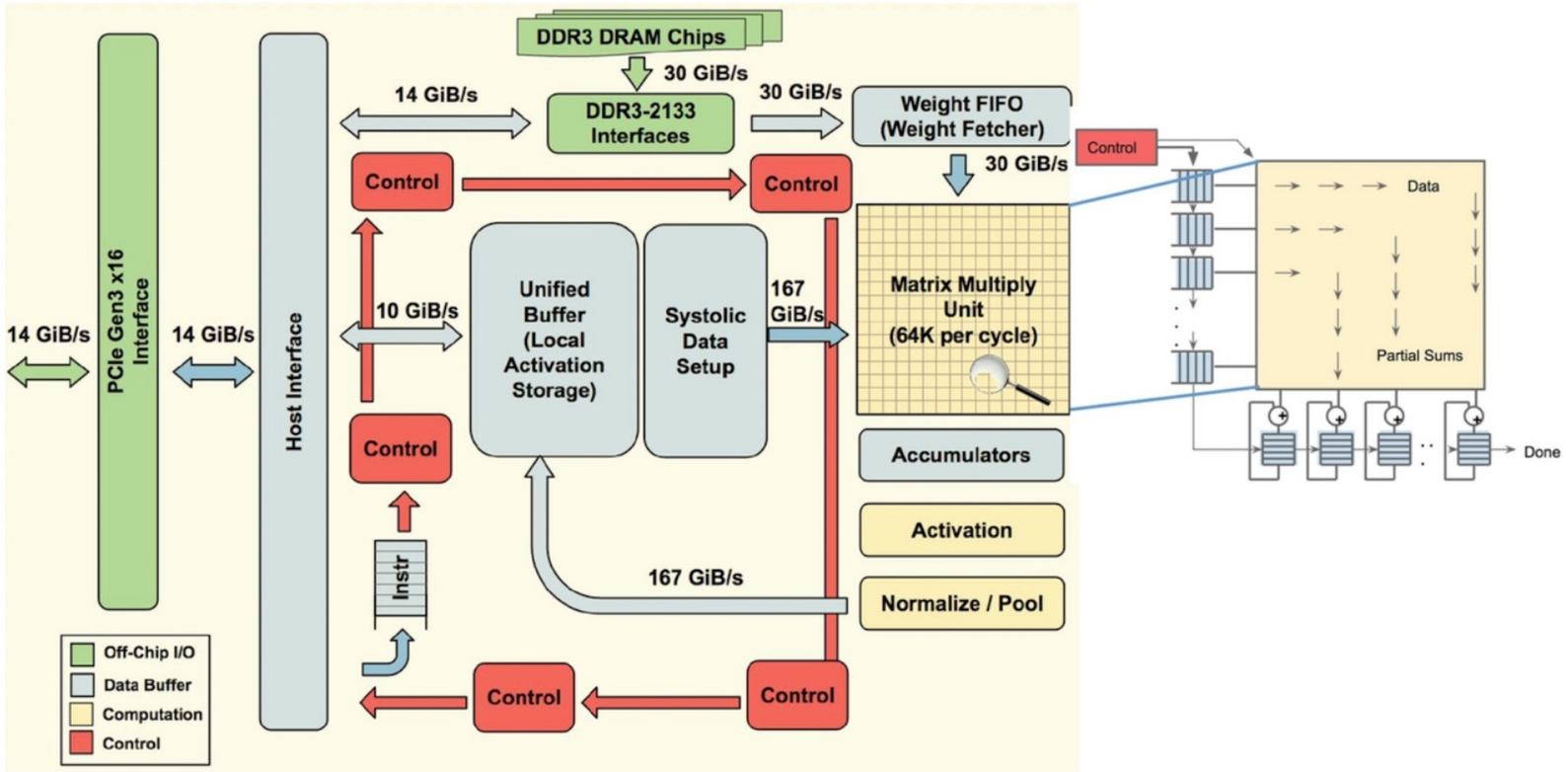
- The "No Free Lunch" (NFL) theorem in optimization and machine learning states that, when averaged across all possible problems, all optimization algorithms perform equally well. I.e., **no algorithm consistently outperforms random search or any other algorithm.**
- The NFL theorem was formalized by David Wolpert and William Macready in 1997.
- **This principle has profound implications for ML and HW optimization:**
  - It explains why specialized algorithms/architectures work well for specific domains but fail in others.
  - It highlights the importance of incorporating domain knowledge when selecting algorithms/architectures.
  - It reminds us that claims about algorithm/architecture superiority must specify the context/problem class.



https://www.geeksforgeeks.org/what-is-no-free-lunch-theorem

https://ieeexplore.ieee.org/document/585893

Architecture of a TPU accelerator

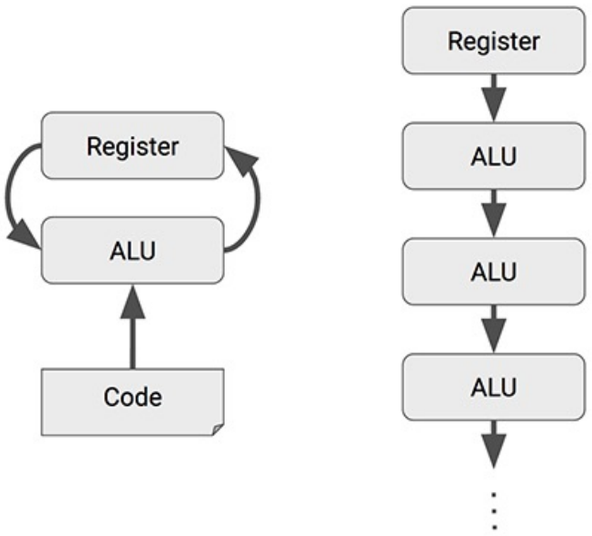


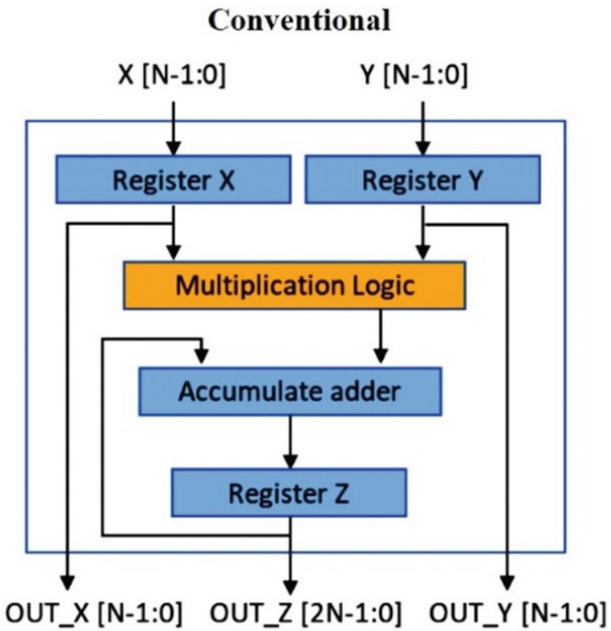
- The main component of TPU is the MXU that consists of 256 by 256 (i.e., 65,536) MACs to perform 8-bit mathematical operations.
- The MXU gets its input from the weighted FIFO and unified buffer (UB).

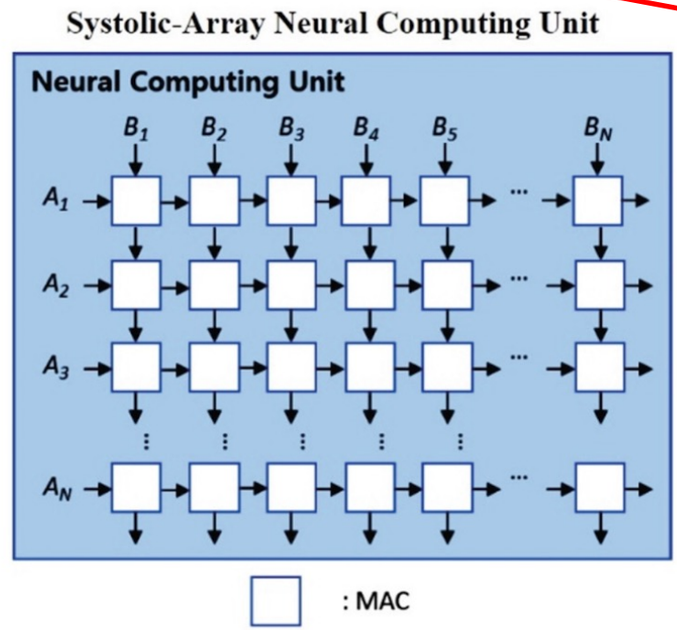
https://cloud.google.com/blog/products/ai-machine-learning/an-in-depth-look-at-googles-first-tensor-processing-unit-tpu

Systolic array

QUIZ



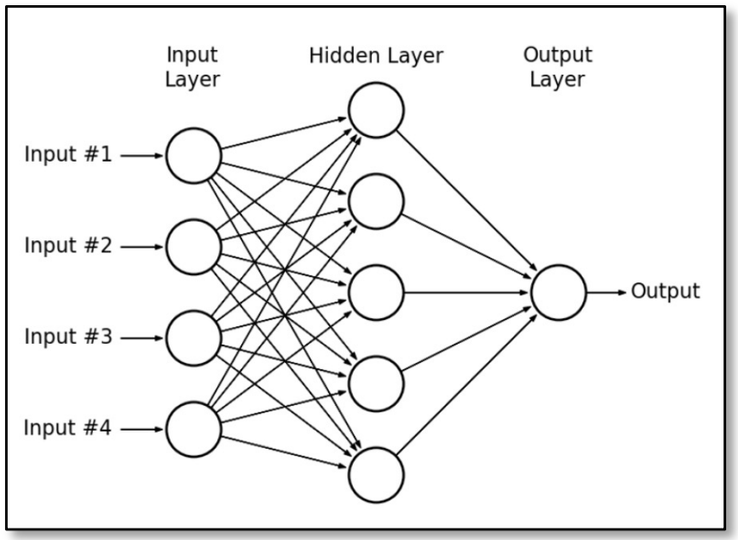


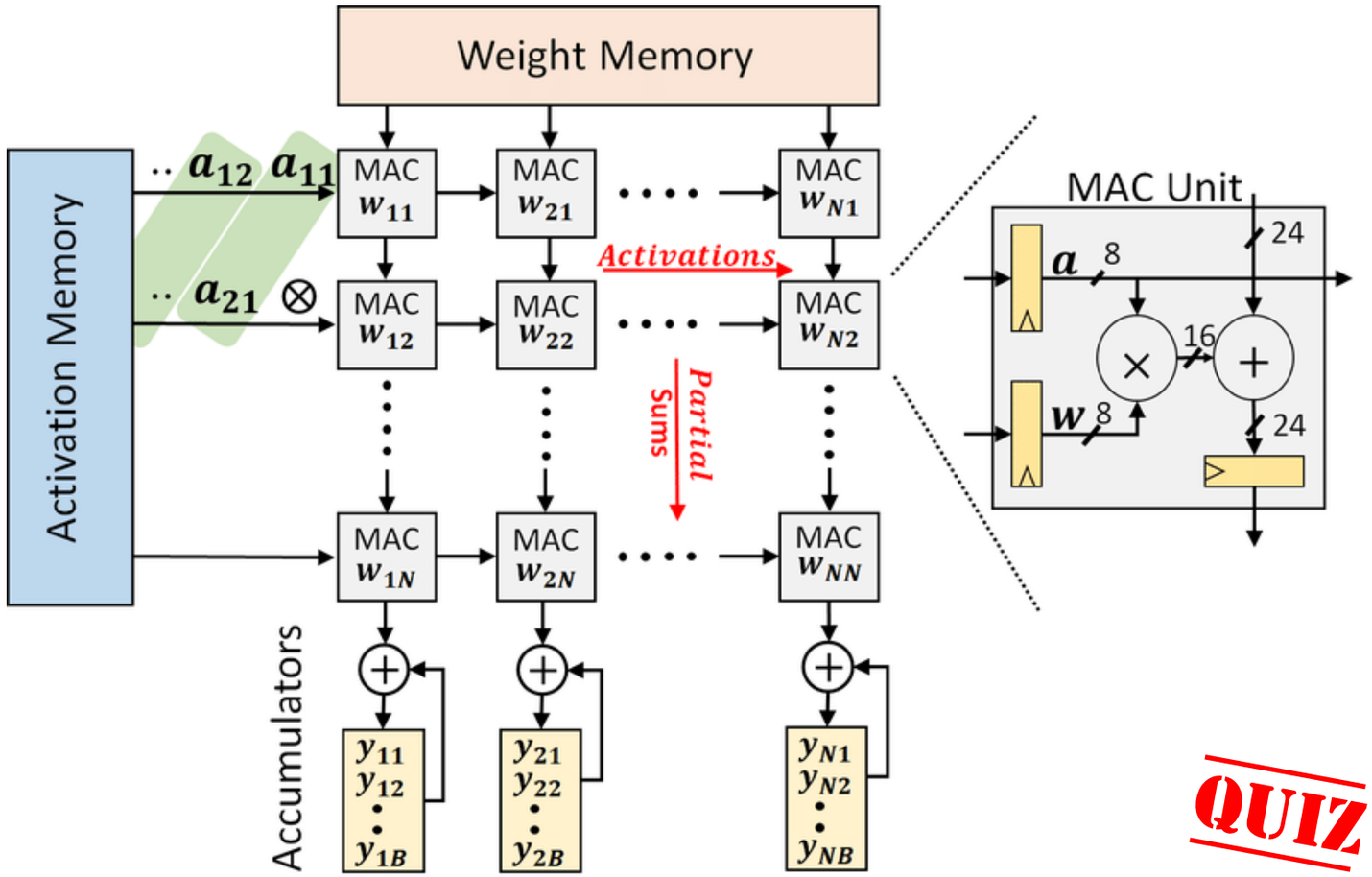


- The name "systolic" comes from an analogy to the human heart, where data pulses through the array of processors in a rhythmic, synchronized fashion.
- CPUs and GPUs often spend energy to access multiple registers per operation. A systolic array chains multiple ALUs together, reusing the result of reading a single register.

Mishra et al., 2023

How to map a deep neural net on a systolic array





QUIZ

https://doi.org/10.1145/3195970.3196129

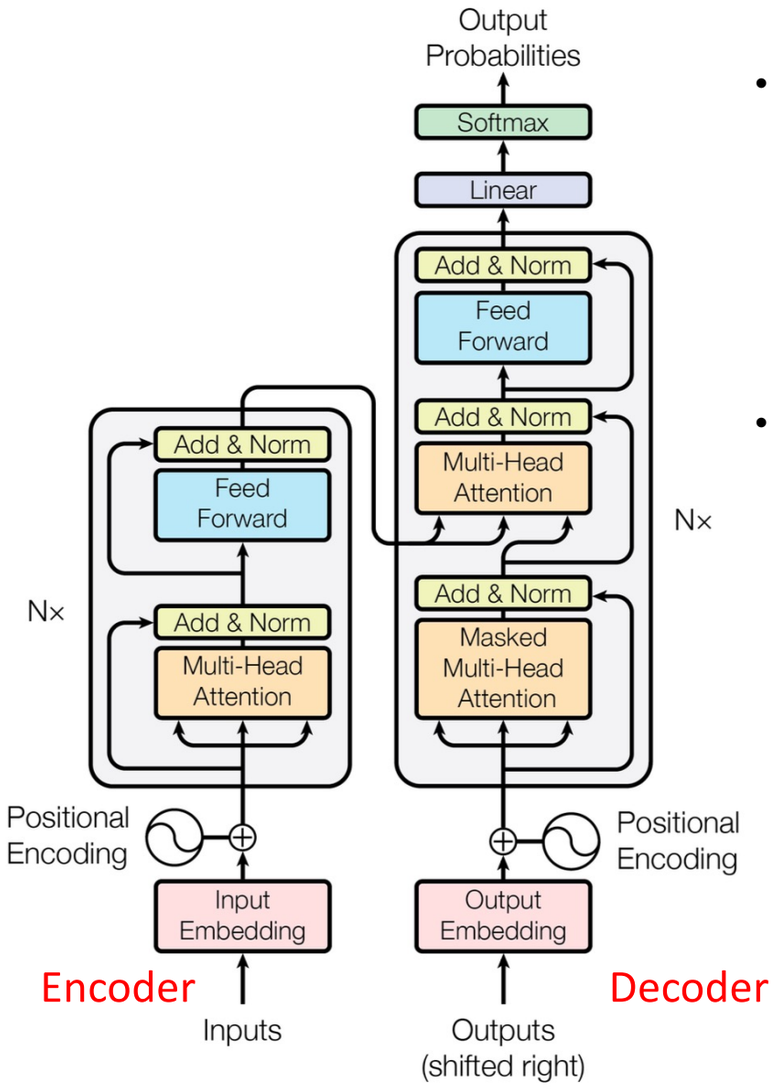
Transformers

Christof Teuscher • teuscher@pdx.edu

What is a transformer?

"A transformer model is a neural network that learns context and thus meaning by **tracking relationships in sequential data** like the words in this sentence." **Transformers made self-supervised learning possible.**

https://arxiv.org/pdf/1706.03762



- "Transformer models apply an evolving set of mathematical techniques, called **attention** or **self-attention**, to detect subtle ways even distant data elements in a series influence and depend on each other."
- Self-attention is asking, for every token: **"which other tokens in this sequence are most relevant to understanding me?"** It does this by learning **three projections** for each token, i.e., Query (Q), Key (K), and Value (V):
  - Q: "What am I looking for?"
  - K: "What do I offer?"
  - V: "What do I actually contribute if selected?"

teuscher Lab
teuscher-lab.com

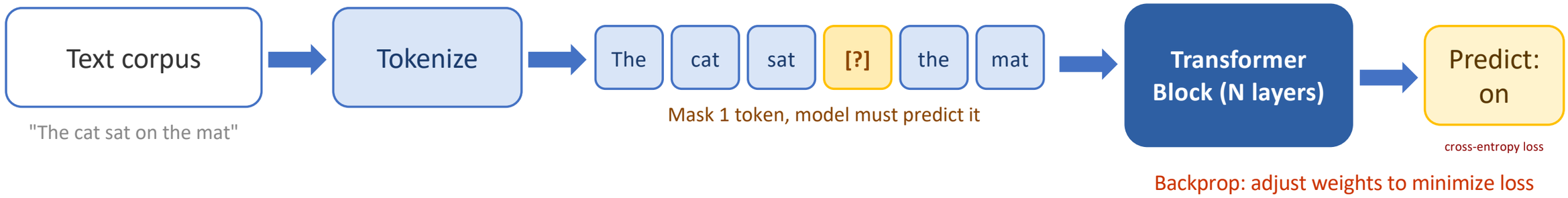
Portland State UNIVERSITY

Christof Teuscher • teuscher@pdx.edu

www.teuscher-lab.com/teaching

How does a transformer learn?

SELF-SUPERVISED PRE-TRAINING: Causal Language Modeling (Next-Token Prediction)



WHAT THE MODEL LEARNS: no human labels required

Syntax and Grammar	World Knowledge	Reasoning Patterns
Word order, agreement, parts of speech	Facts, entities, common sense from text	Inference, analogy, cause and effect

The next word in the text is always the label. Billions of free training signals from any corpus, no annotation required. Transformers learn syntax, world knowledge, and reasoning purely by predicting what comes next.

Christof Teuscher • teuscher@pdx.edu

www.teuscher-lab.com/teaching

RL-Based Fine-Tuning to Align Transformers (RLHF)

RL-BASED FINE-TUNING: Reinforcement Learning from Human Feedback (RLHF)

1 Supervised Fine-Tuning (SFT)

Pretrained LLM → Fine-tune on demos

Human demos: prompt + ideal response

SFT Model

Learns format, tone, helpfulness

Output: initialized policy

Teaches model what good looks like

2 Reward Model Training

SFT generates multiple responses per prompt

Response A | Response B

Humans rank: A preferred over B

Reward Model (RM)

Cross-entropy trained on ranked pairs

Outputs scalar score r(x,y)

3 PPO RL Fine-Tuning

Policy generates; RM scores; PPO updates

Policy LLM → Response y → **RM score: r**

Max E[r(x,y)] - b*KL(policy, SFT)

KL penalty prevents reward hacking

Repeat until model is aligned

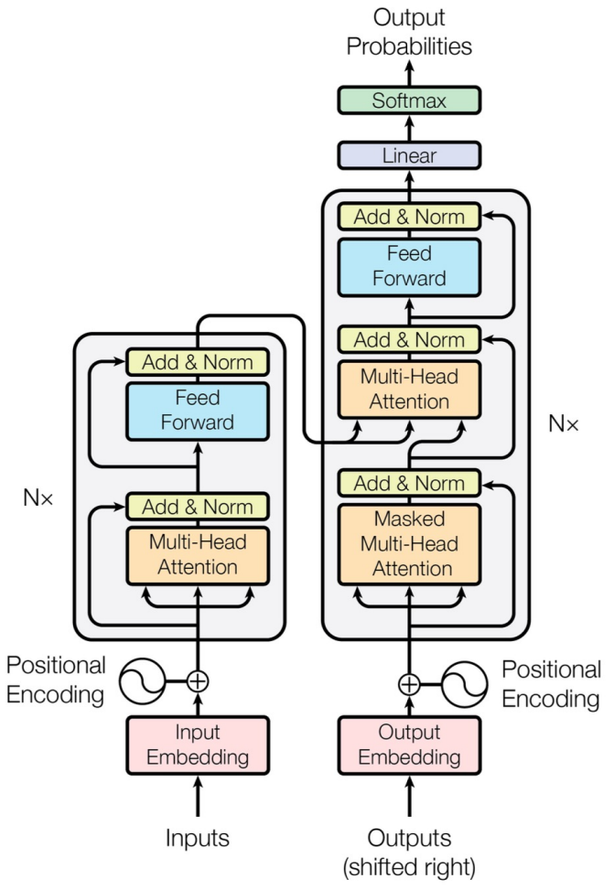
Humans rank outputs more easily than write perfect ones. RLHF converts preferences into reward signals, steering models toward helpful, harmless, honest behavior.

teuscher Lab
teuscher-lab.com

PPO = Proximal Policy Optimization
RM = Reward Model
KL = Kullback-Leibler divergence

Portland State UNIVERSITY

What is recurrent in a transformer?



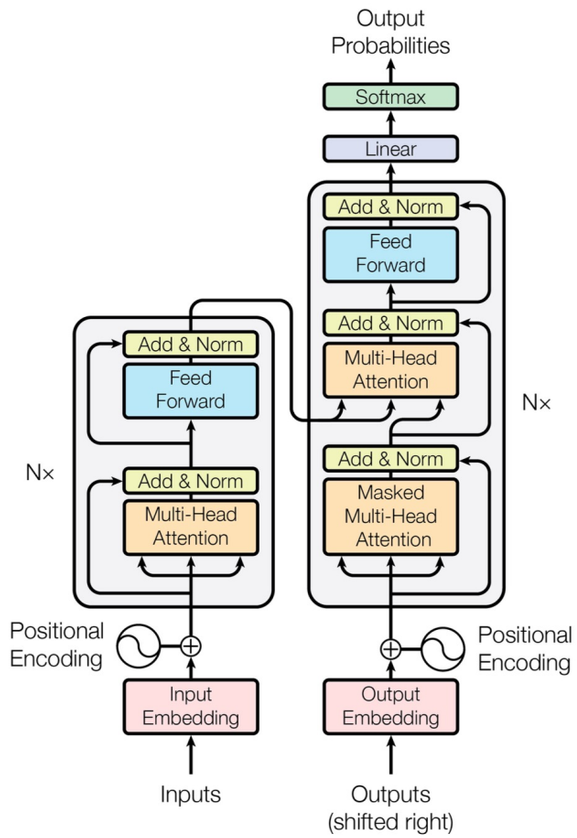
- **Transformers are explicitly non-recurrent.**
- The "Attention Is All You Need" paper (Vaswani et al., 2017) was a direct response to RNNs/LSTMs, which are recurrent. RNNs/LSTMs process tokens one at a time, passing a hidden state from step to step. That sequential dependency is what makes RNNs slow to train (can't parallelize across time steps) and bad at long-range dependencies (the hidden state bottleneck).

- **The Transformer replaces recurrence entirely with self-attention, which**:
  - Processes all tokens simultaneously in parallel
  - Connects every token directly to every other token in a single operation, i.e., no hidden state to propagate
  - Has no notion of "previous step," positional encodings handle order instead (time is replaced by position).

https://arxiv.org/pdf/1706.03762
https://blogs.nvidia.com/blog/what-is-a-transformer-model

What is a transformer?

- Addresses all drawbacks of recurrent neural nets (RNN)
- No labels (costly and time-consuming)
- Very parallelizable
- Higher performance
- Like most neural networks, transformer models are basically large **encoder/decoder (compression/decompression)** blocks that process data.
- Transformers use **positional encoders** to tag data elements coming in and out of the network.
- **Attention units** follow these tags, calculating a kind of algebraic map of how each element relates to the others.
- **Attention queries** are typically executed in parallel by calculating a matrix of equations in what's called **multi-headed attention**.



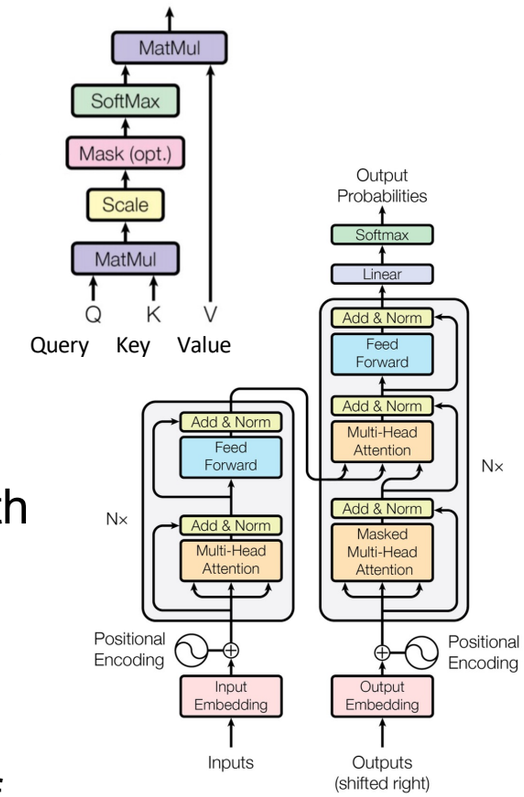
https://arxiv.org/pdf/1706.03762
https://blogs.nvidia.com/blog/what-is-a-transformer-model

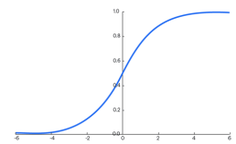
Christof Teuscher • teuscher@pdx.edu

Main mathematical operations of a transformer

- **Matrix Multiplication** - Used throughout the architecture, particularly in the attention mechanism and feed-forward networks.
- **Attention Mechanism** - The core operation involving:
  - Query, Key, Value transformations (matrix multiplications)
  - Scaled dot-product attention: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$
  - Multi-head attention which runs multiple attention operations in parallel
- **Layer Normalization** - Normalizes the outputs of sub-layers using mean and variance statistics
- **Residual Connections** (Skip Connections) - Addition operations that help with gradient flow
- **Position-wise Feed-Forward Networks** - Two linear transformations with a ReLU activation in between
- **Softmax** - Used in the attention mechanism to convert scores to probabilities
- **Positional Encoding** - Often implemented using sine and cosine functions of different frequencies [GPU SFU]
- **Embedding** - Converting tokens to continuous vector representations

[GPU SFU]





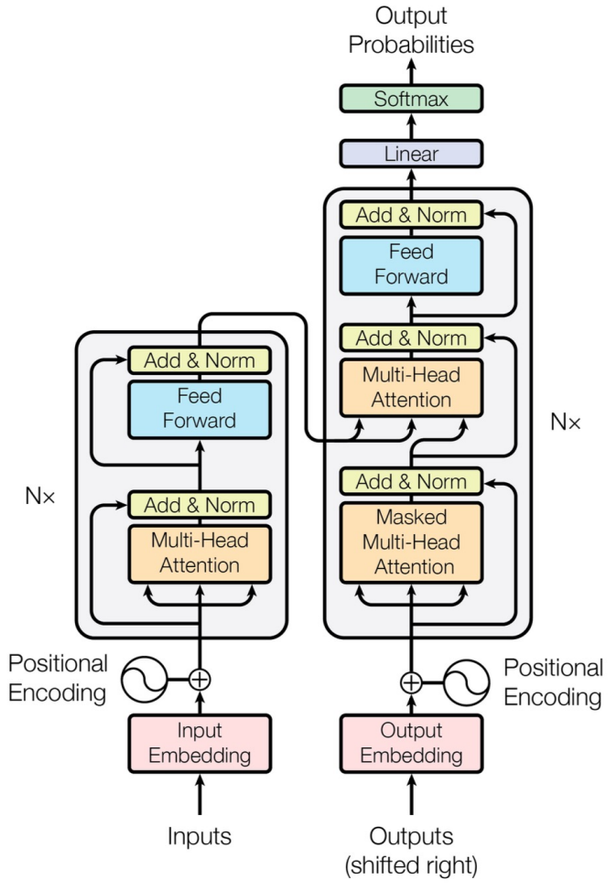
$$s\left(x_i\right) = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$$

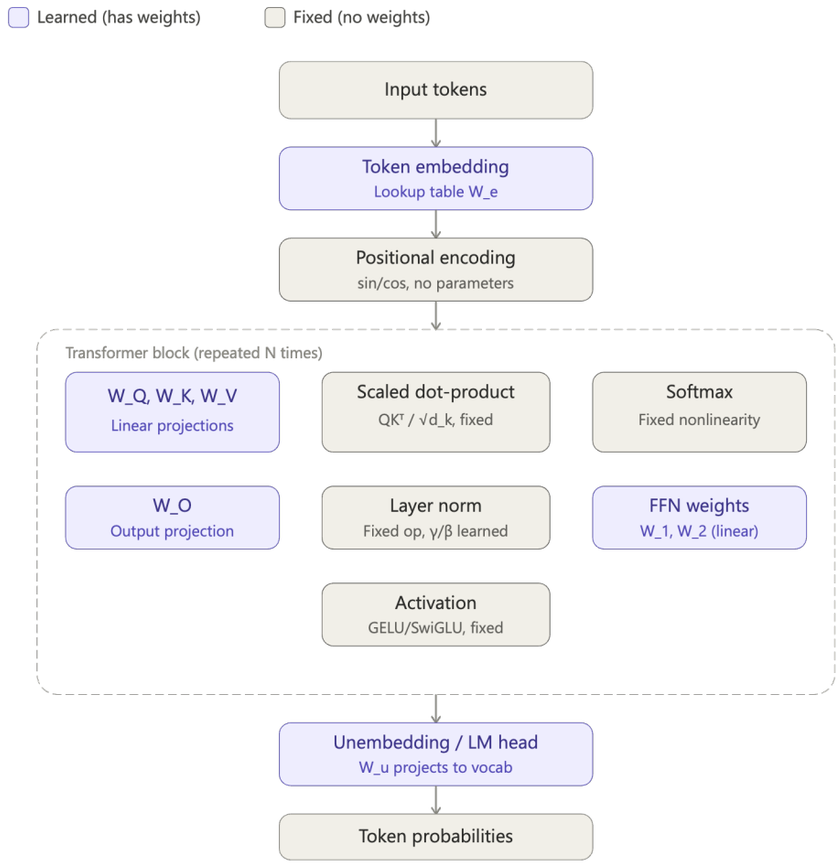
Softmax is a function that converts a vector of arbitrary real numbers into a probability distribution. All outputs are positive and sum to exactly 1.

teuscher Lab
teuscher-lab.com

Portland State UNIVERSITY

What learns, what is fixed?





Christof Teuscher • teuscher@pdx.edu

www.teuscher-lab.com/teaching

How to map transformers on a GPU

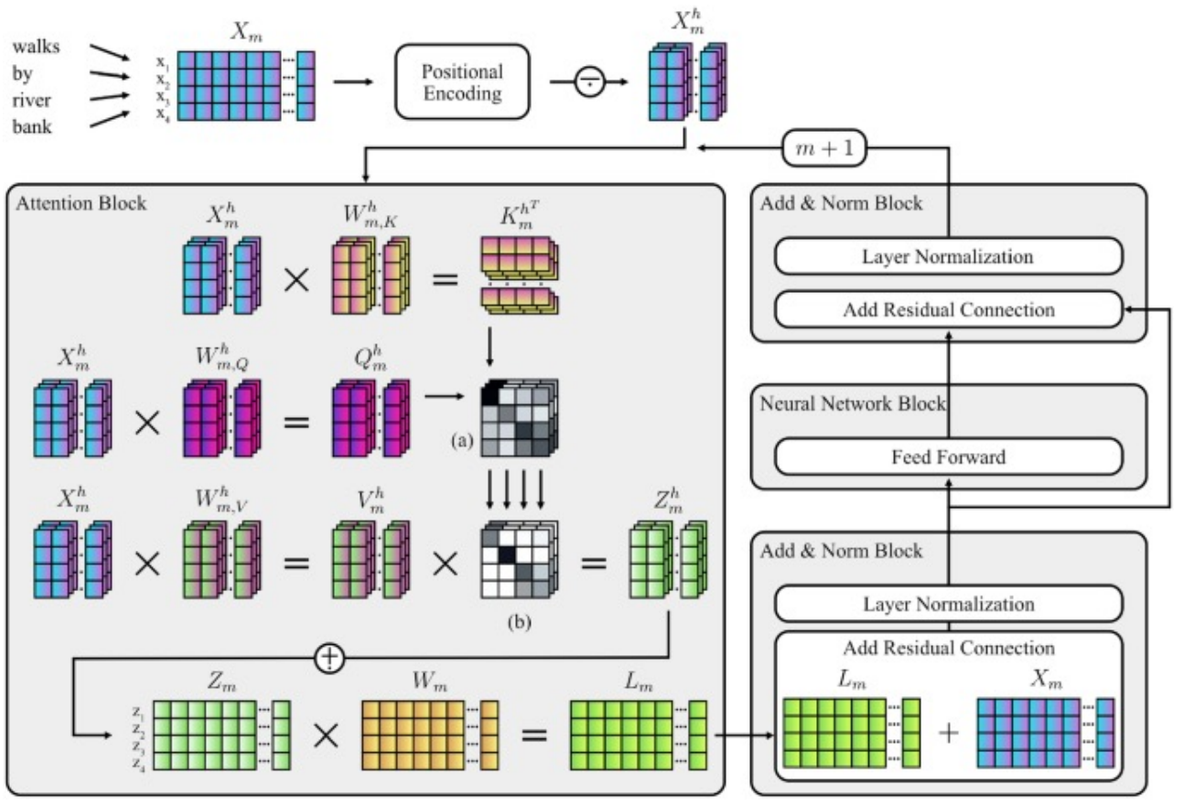


FIGURE 1.
Schematic Diagram of the Attention-Mechanism and Components of the Transformer Architecture. *Note*. The process illustrates the encoding and transformation of the sequence "*walks by river bank*" by components of the transformer architecture (Vaswani et al., 2017). Weight matrices ($W^h_{m,\,K|Q|V}$ and W_m) are randomly initialized and then learned during the training process. In case of causal language models, masking (see Eq. 5) is applied to Z^h_m. (a) = Matrix product of $K^{h^T}_m$ and Q^h_m; (b) Scaling and softmax is applied; n = Input sequence length; d = Model dimensionality, i.e., length of embedding vectors; h = Current attention head; n_h = Number of attention heads; m = Current layer; X_m = Embedding matrix ($dimensionality : n \times d$); X^h_m = Embedding matrix subset ($n \times \frac{d}{n_h}$); $W^h_{m,\,K|Q|V}$ = Key, query, and value weight matrices ($n \times \frac{d}{n_h}$); $K^{h^T}_m$ = Transposed key matrix ($n \times \frac{d}{n_h}$); Q^h_m = Query matrix ($n \times \frac{d}{n_h}$); V^h_m = Value matrix ($n \times \frac{d}{n_h}$); Z_m = Attention matrix ($n \times d$); W_m = Weight matrix ($n \times d$); L_m = Layer output matrix ($n \times d$); $\dot{-}$ = Matrix subdivision; $\dot{+}$ = Matrix concatenation.

- Q: "What am I looking for?"
- K: "What do I offer?"
- V: "What do I actually contribute if selected?"

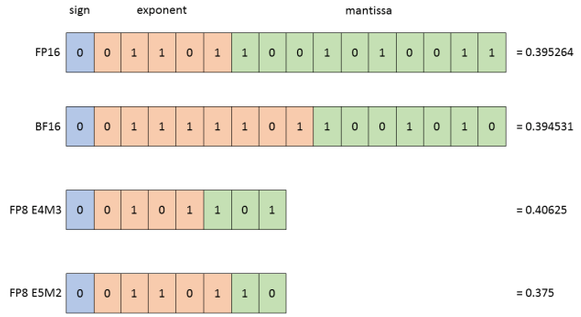
Hommel, B.E., Wollang, FJ.M., Kotova, V. et al. **Transformer-Based Deep Neural Language Modeling for Construct-Specific Automatic Item Generation**. Psychometrika 87, 749–772 (2022). https://doi.org/10.1007/s11336-021-09823-9

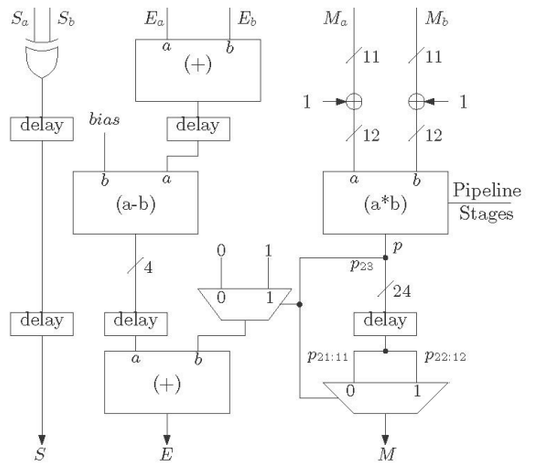
teuscher Lab
teuscher-lab.com

Portland State UNIVERSITY

REMINDER: Why lower precision?

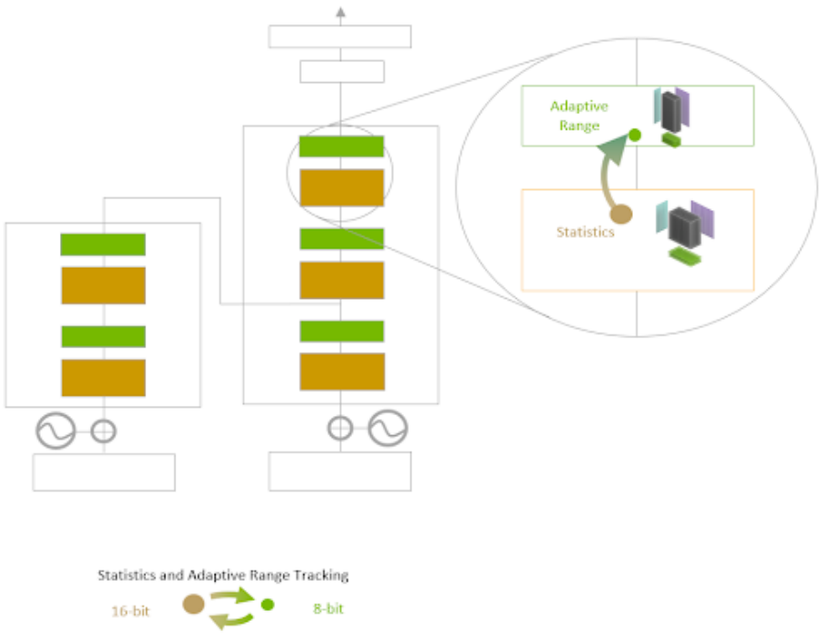
- **Data Transfer Efficiency**: FP8 values are half the size of FP16; FP4 halves again. More data per memory transaction.
- **Cache Efficiency**: Smaller types → more values fit in cache, reducing expensive off-chip memory accesses.
- **Compute Density**: More arithmetic units in the same silicon area → more parallel operations per cycle.
- **Power Efficiency**: Lower precision consumes less power, allowing higher clocks or more ops per watt.
- **FP4 (NEW — 2024+)**: NVIDIA Blackwell (B200) introduces FP4 via 2nd-gen Transformer Engine — 20 PFLOPS sparse; Google's Ironwood (TPU v7) adds native FP8. Microscaling formats (MXFP4/MXFP6) maintain accuracy at 4-bit.

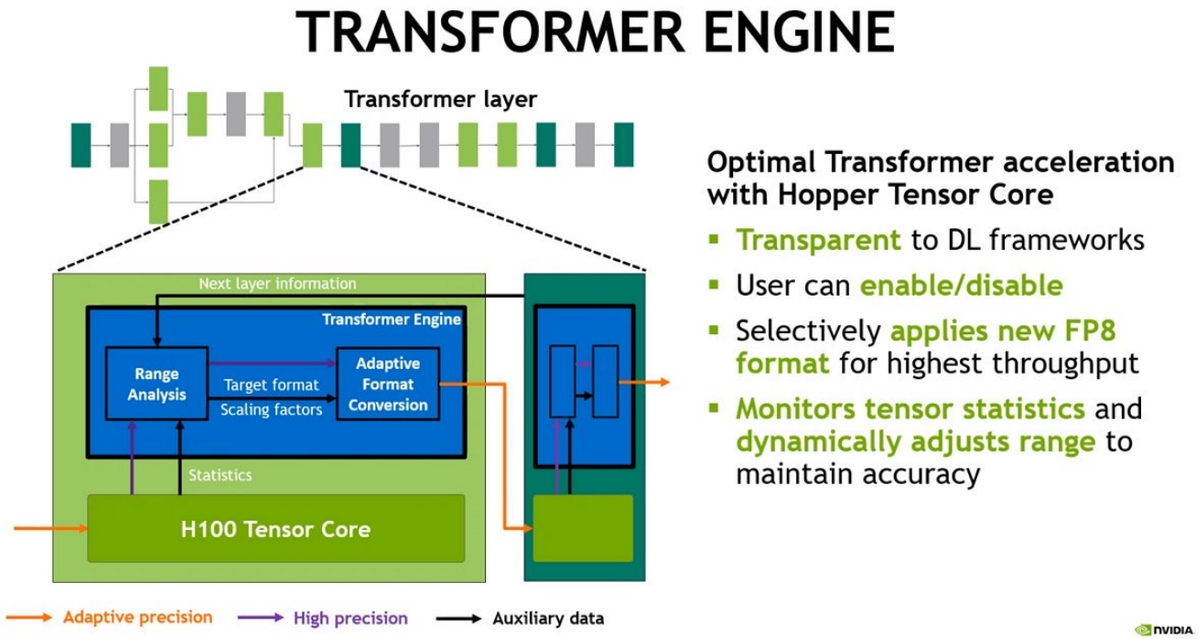




NVIDIA Transformer Engine (1st gen)

Essentially a library that uses the the GPU tensor cores.





Transformer Engine uses per-layer statistical analysis to **dynamically determine the optimal precision (FP16 or FP8) for each layer of a model**, achieving the best performance while preserving model accuracy.

https://www.hardwarezone.com.sg/tech-news-nvidia-h100-gpu-hopper-architecture-building-block-ai-infrastructure-dgx-h100-supercomputer
https://blogs.nvidia.com/blog/h100-transformer-engine
https://docs.nvidia.com/deeplearning/transformer-engine/user-guide/index.html
https://www.nvidia.com/en-us/data-center/technologies/blackwell-architecture

Christof Teuscher • teuscher@pdx.edu

NVIDIA Blackwell B200: Key Specifications

Specification	H100 (Hopper)	B200 (Blackwell)
Process	TSMC 4N	**TSMC 4NP**
Transistors	80 B	**208 B (dual-die)**
HBM Capacity	80 GB HBM3	**192 GB HBM3e**
Memory Bandwidth	3.35 TB/s	**8 TB/s**
Tensor Cores	4th gen	**5th gen**
Lowest Precision	FP8	**FP4 (MXFP4)**
Transformer Engine	1st gen	**2nd gen (FP4)**
Peak AI (sparse)	~4 PFLOPS FP8	**20 PFLOPS FP4**
NVLink	4.0 (900 GB/s)	**5.0 (1.8 TB/s)**
TDP	700 W	**1000 W**

What's New in Blackwell

- Dual-die design: Two GB100 dies connected by 10 TB/s interconnect
- FP4 Tensor Cores: 2× throughput vs FP8 for inference; MXFP4 format preserves accuracy via per-block scaling
- 2nd-gen Transformer Engine: Auto mixed-precision between FP4/FP8/BF16 per layer
- NVLink 5: 1.8 TB/s bidirectional — removes PCIe bottleneck for multi-GPU
- GB200 NVL72: 72-GPU rack-scale system; 30× faster LLM inference vs H100

Sources: nvidia.com/blackwell; Wikipedia: Blackwell (microarchitecture)

Journals & Magazines > IEEE Transactions on Circuits... > Early Access

A Neuromorphic Transformer Architecture Enabling Hardware-Friendly Edge Computing

Publisher: IEEE Cite This PDF

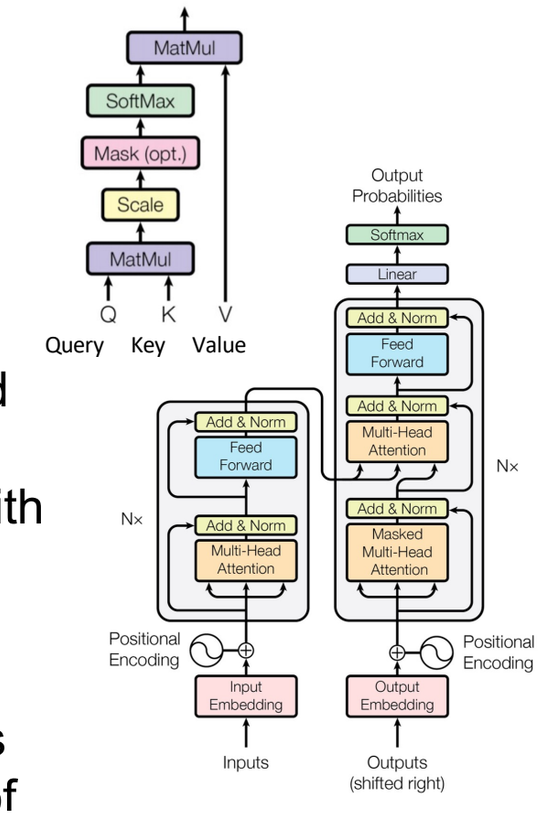
P. J. Zhou ; R. C. Ma ; Y. C. Chen ; Z. T. Liu ; C. Y. Liu ; L. W. Meng **All Authors**

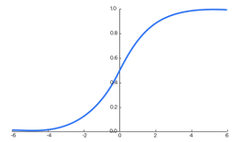
Abstract:
The transformer model has demonstrated significant capabilities in various intelligent tasks, attracting widespread attention in recent years. However, it involves numerous complex operations, including large-bit-width multiplication, division, matrix transposition, and exponentiation. These require substantial storage and computational resources, making it challenging to deploy on edge devices.

https://ieeexplore.ieee.org/abstract/document/10962199

What transformer operations could we improve?

- **Matrix Multiplication** - Used throughout the architecture, particularly in the attention mechanism and feed-forward networks.
- **Attention Mechanism** - The core operation involving:
  - Query, Key, Value transformations (matrix multiplications)
  - Scaled dot-product attention: $\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$
  - Multi-head attention which runs multiple attention operations in parallel
- **Layer Normalization** - Normalizes the outputs of sub-layers using mean and variance statistics
- **Residual Connections** (Skip Connections) - Addition operations that help with gradient flow
- **Position-wise Feed-Forward Networks** - Two linear transformations with a ReLU activation in between
- **Softmax** - Used in the attention mechanism to convert scores to probabilities
- **Positional Encoding** - Often implemented using sine and cosine functions of different frequencies
- **Embedding** - Converting tokens to continuous vector representations

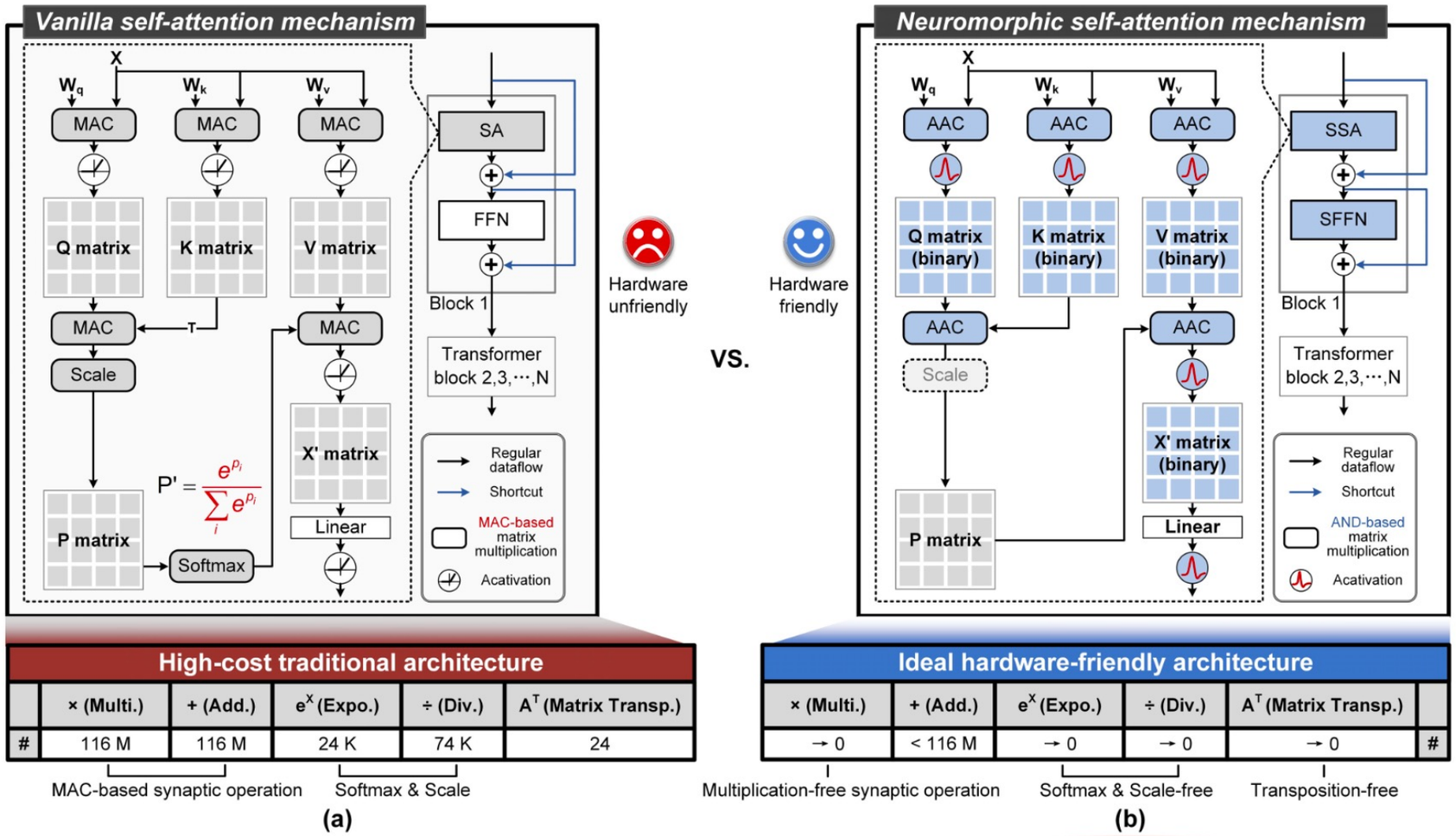




$$s\left(x_i\right) = \frac{e^{x_i}}{\sum_{j=1}^{n} e^{x_j}}$$

Neuromorphic self-attention mechanism

FFN = Feed-Forward Network
SA = self-attention



High-cost traditional architecture

	× (Multi.)	+ (Add.)	e^x (Expo.)	÷ (Div.)	A^T (Matrix Transp.)
#	116 M	116 M	24 K	74 K	24

MAC-based synaptic operation — Softmax & Scale

(a)

Ideal hardware-friendly architecture

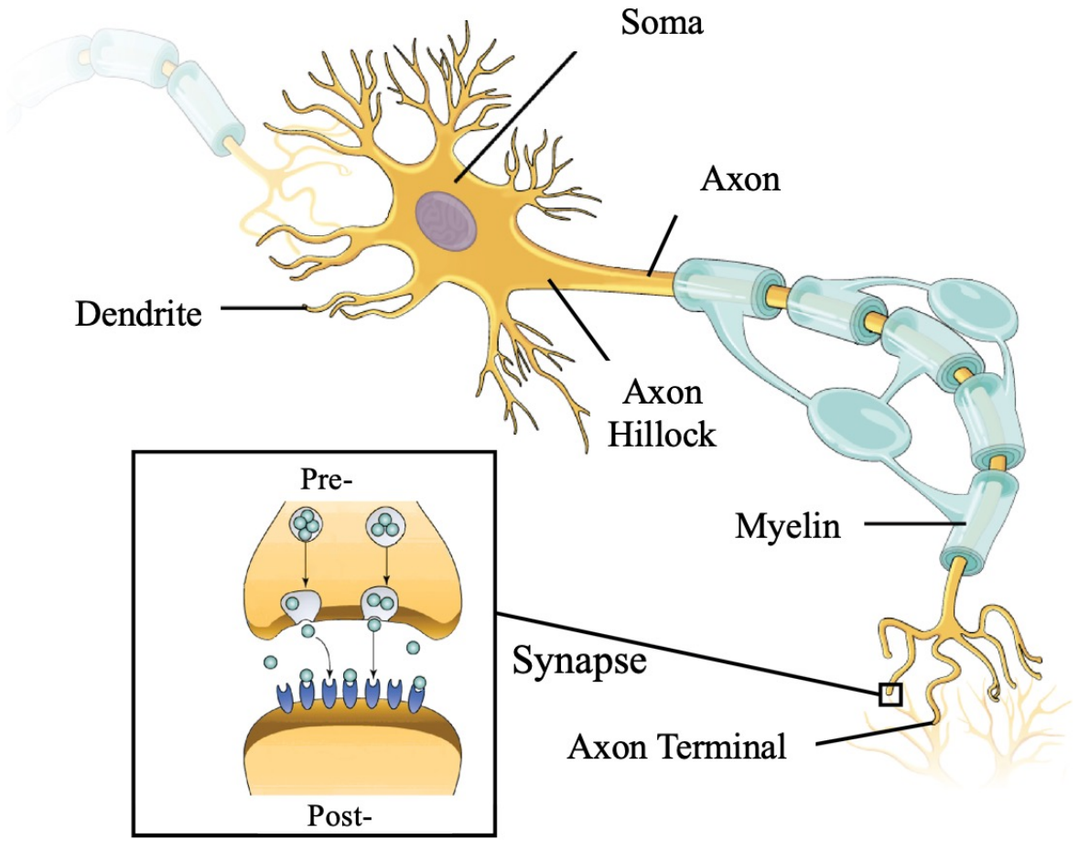
× (Multi.)	+ (Add.)	e^x (Expo.)	÷ (Div.)	A^T (Matrix Transp.)	
→ 0	< 116 M	→ 0	→ 0	→ 0	#

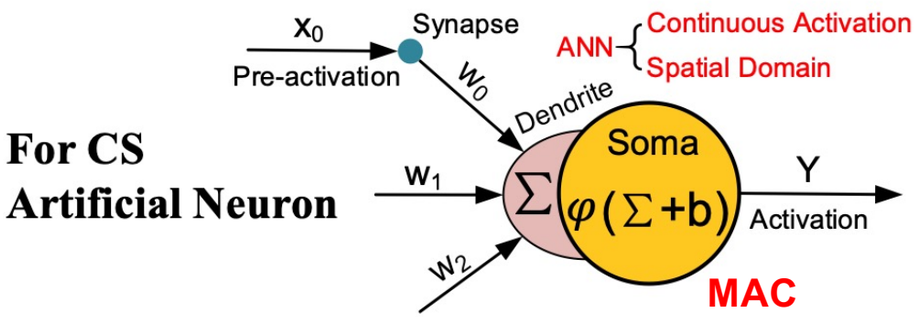
Multiplication-free synaptic operation — Softmax & Scale-free — Transposition-free

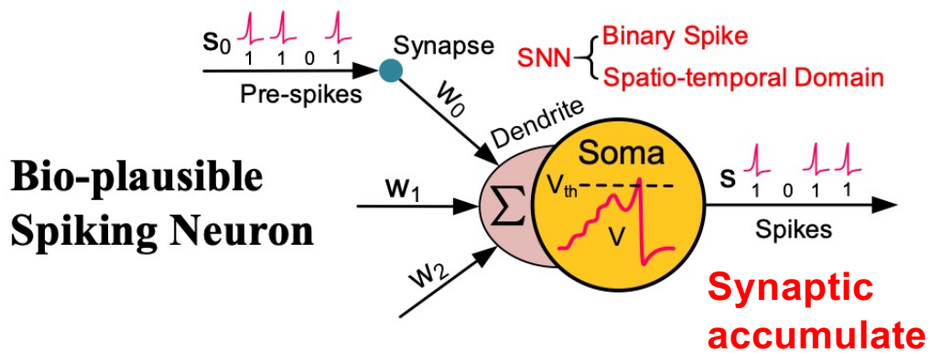
(b)

https://ieeexplore.ieee.org/abstract/document/10962199

To spike or not to spike (1)







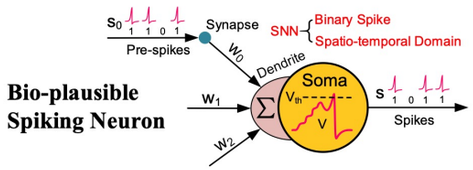
Advanced SNN = ANN + Neuronal Dynamics

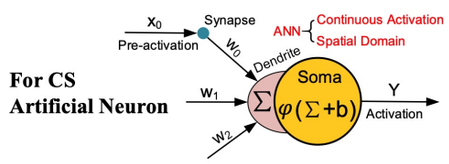
Christof Teuscher • teuscher@pdx.edu

To spike or not to spike (2)

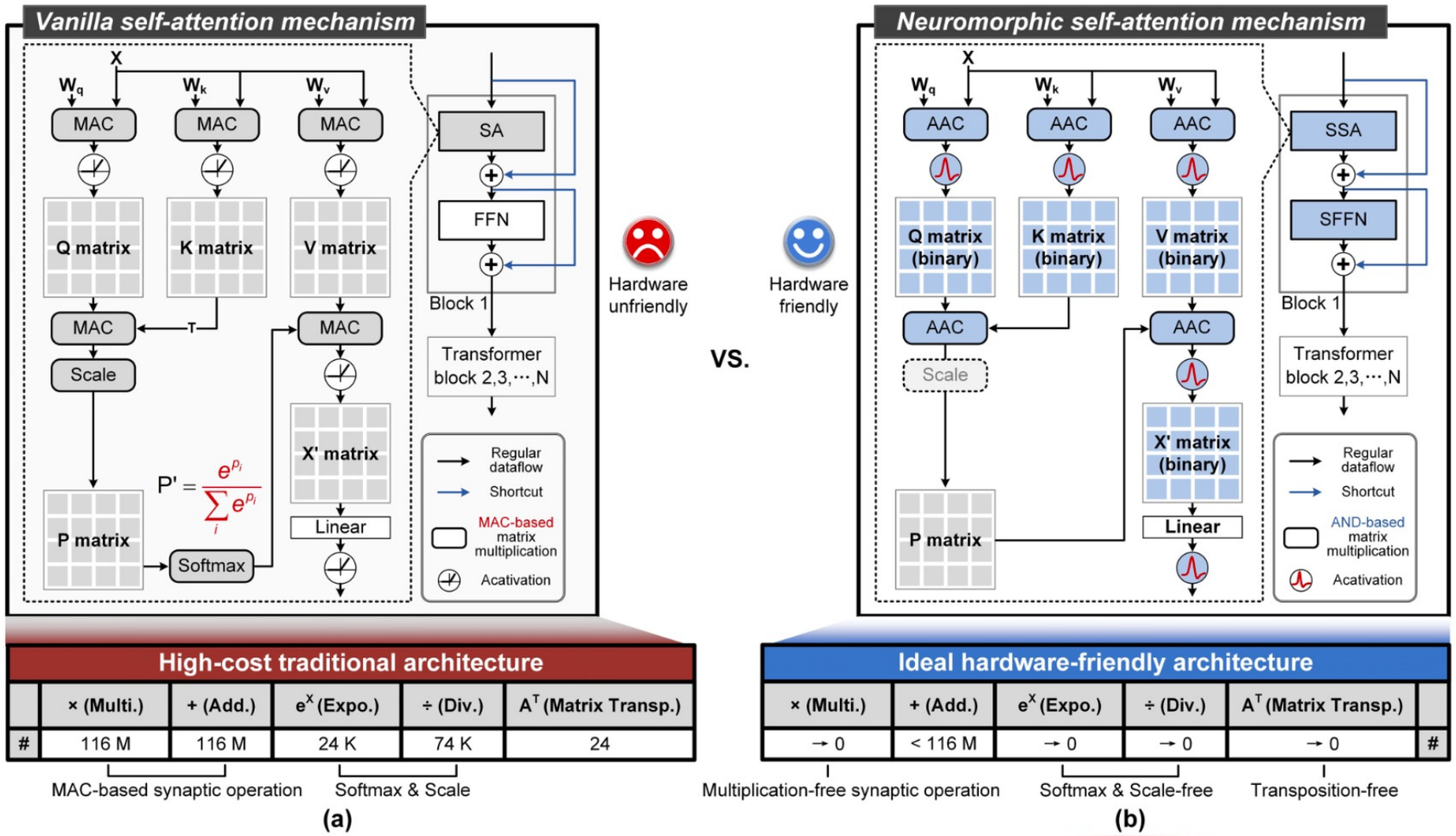
Characteristic	Spiking Neural Networks (SNNs)	Analog Neural Networks (ANNs)
Information Encoding	Discrete spikes/events (binary 0/1 signals)	Continuous values (floating-point)
Biological Plausibility	High - mimics actual neuron behavior	Low - simplified abstraction
Temporal Dynamics	Inherently time-based processing	No native temporal processing
Energy Efficiency	Potentially very high (event-driven)	Moderate to high (depends on implementation)
Training Methods	STDP, converted ANN weights, specialized algorithms	Backpropagation, gradient descent
Accuracy (Current)	Lower but improving	State-of-the-art for most tasks
Hardware Implementation	Specialized neuromorphic chips (Loihi, TrueNorth, SpiNNaker)	GPUs, TPUs, traditional computing hardware
Computational Model	Asynchronous, event-driven	Synchronous, clock-driven
Sparsity	Naturally sparse (neurons only fire when threshold reached)	Dense computations (all neurons compute at each step)
Neuron Models	Leaky integrate-and-fire, Hodgkin-Huxley, Izhikevich	Simple activation functions (ReLU, sigmoid, tanh)
Best Application Domains	Real-time processing, temporal data, power-constrained devices	General ML tasks, image classification, NLP
Maturity	Emerging technology, active research area	Mature technology, widely deployed

teuscher Lab
teuscher-lab.com





Neuromorphic self-attention mechanism



AAC = AND accumulate

Why is that equivalent to MAC?

https://ieeexplore.ieee.org/abstract/document/10962199

AND accumulate

Traditional:

`output = (input1 × weight1) + (input2 × weight2) + ... + (inputN × weightN)`

AND accumulate:

`output = (input1 AND weight1) + (input2 AND weight2) + ... + (inputN AND weightN)`

This process essentially counts how many input-weight pairs are both "on" (1) at the same time, which serves as an approximation of multiplication in binary or highly quantized networks.

Binary multiplication = AND operation

Architecture

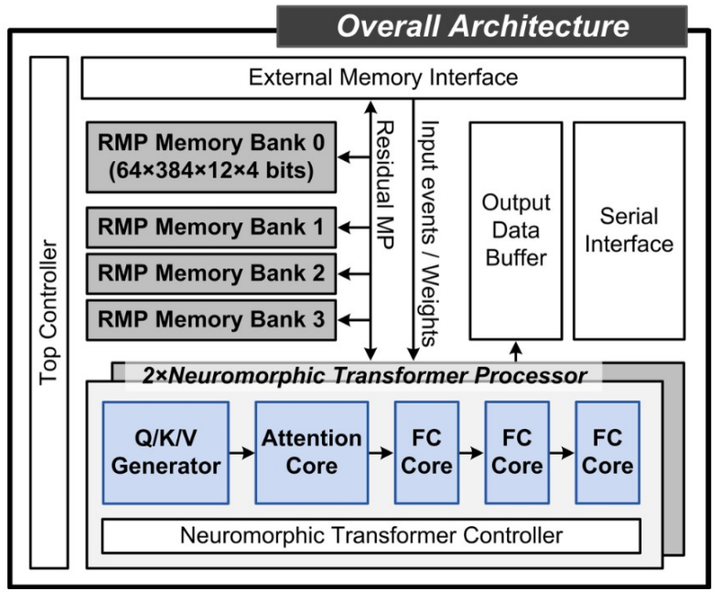


Fig. 6. The overall architecture mainly consists of two neuromorphic transformer processors and peripheral modules.

FC = Fixed-Function Core

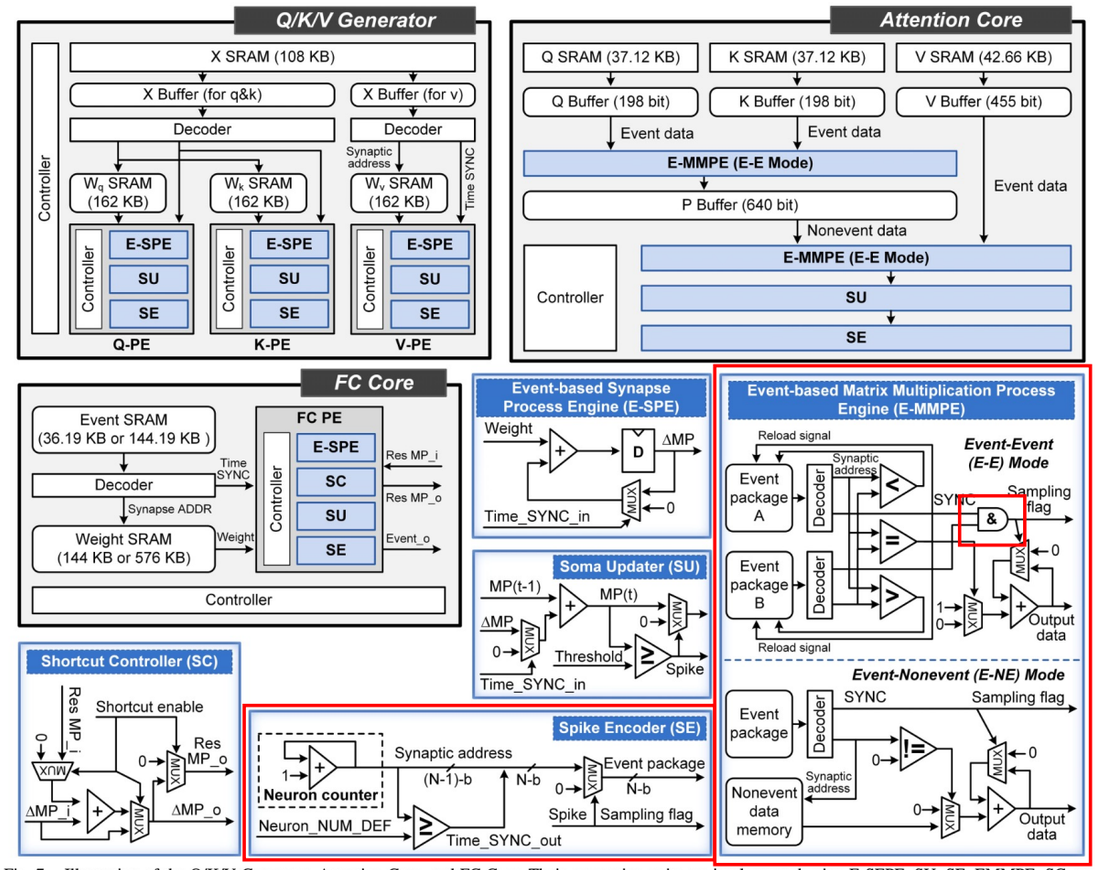
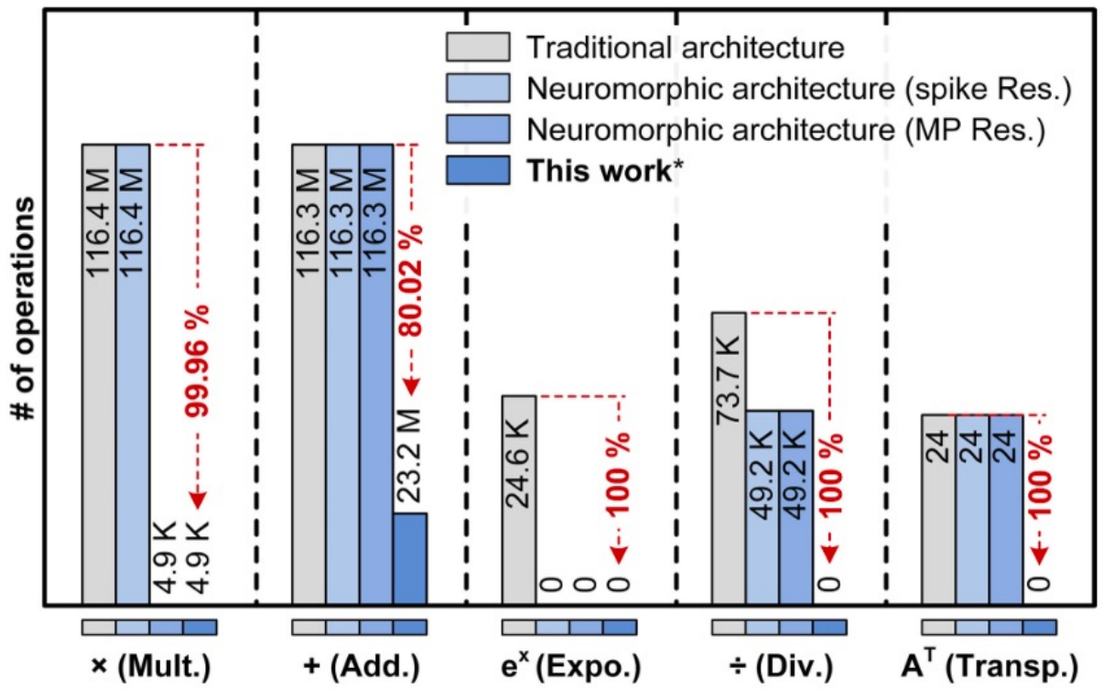


Fig. 7. Illustration of the Q/K/V Generator, Attention Core, and FC Core. Their computing units are implemented using E-SEPE, SU, SE, EMMPE, SC, etc.

https://ieeexplore.ieee.org/abstract/document/10962199

Resource overhead



* Two transformer blocks with 80% spike sparsity: MP Res, scaling factor absorption, transposition calculation, spike-driven synaptic computing.

Fig. 8. Computational resource overhead of different transformer architectures.

Res. = Residuals
MP = membrane potential

https://ieeexplore.ieee.org/abstract/document/10962199

Comparison with state-of-the-art

	JSSC22 [18]	JSSC23 [19]	JSSC24 [48]	TBCAS19 [56]	JSSC24 [43]	NSR24 [57]	**This work**
Architecture	ANN (4×systolic-based attention core)	ANN (1×systolic-based general core)	ANN&SNN (64×FC core)	SNN (4×FC core)	SNN (1024×FC core)	SNN (575×FC core)	**SNN (2×transformer block)**
Technology (nm)	28	65	28	65	28	22	**28**
Die area (mm^2)	6.82	6.40	1.63	3.5	537.98	358.53	**23.28 (all-mem on chip)**[a]
Frequency (MHz)	50-510	600-1100	40-210	210	24-600	300-400	**400**
Power (mW)	12.06-272.7	400-1675	2.91-56.8	19.9	9970	1800	**798.59**[b]
On-chip Memory	336 KB	64 KB	266.5 KB	288 KB	181.865 MB	N. A.	**5.13 MB**
Synaptic precision	INT12	INT8	INT8/10	INT1	INT1/2/4/8	INT1/2/4/8/16	**INT8**
Softmax	Required	Required	-	-	-	-	**Unrequired**
Scale	Required	Required	-	-	-	-	**Unrequired**
Matrix transposition	Required	Required	-	-	-	-	**Unrequired**
Synaptic MAC	Required	Required	Required	Unrequired	Unrequired	Unrequired	**Unrequired**
Accuracy (Cifar-10 dataset)	N. A.	91.5% (CNN)	N. A.	N. A.	93.91% (CNN)	90.17% (CNN)	**94.03 %**[c]
Computing efficiency per core[d] (GSOP/s)	65.25 (MM) 65.25-157.5 (QK^T) 65.25-508.75 (PV) (@510 MHz)	512	N. A.	0.105 (@210 MHz)	20.25 (@600 MHz)	N. A.	**G^f: 1.49-2.93 A^f: 1.10-1.84 F^f: 1.00-1.98**
Energy efficiency[e] (pJ/SOP)	1.047 (MM) 0.77-0.32 (QK^T) 0.77-0.14 (PV) (@510 MHz)	3.33-2 (@1 GHz)	4.16 (@210 MHz)	65 (@210 MHz)	2.444 (@120 MHz)	5.4 (@300-400 MHz)	**G^f: 0.64-0.33 A^f: 4.34-2.79 F^f: 5.71-3.07**

SOP = Standard Operations Per Second

GSOP/s = Giga Standard Operations Per Second

Different cores of their design

https://ieeexplore.ieee.org/abstract/document/10962199

What are the biggest bottlenecks in traditional computer architecture?

Impact of major bottlenecks

Bottleneck	Performance Impact	Trend	Mitigation Strategies	Relative Importance
Memory Wall (Von Neumann)	50-80% performance gap between CPU and DRAM speeds	Growing (CPU performance improves ~40% annually vs. memory at ~10%)	Cache hierarchies, HBM, on-chip memory	★★★★★
Power Consumption	~100x increase in power density since 1990s	Exponential growth halted clock speeds at ~4GHz	Dark silicon, specialized cores, power gating	★★★★★
Sequential Processing	Amdahl's Law: 5x parallel cores yields only ~3x speedup with 10% sequential code	Sequential portion becomes dominant constraint	Speculative execution, out-of-order processing	★★★★
I/O Bottlenecks	Storage access ~10^5-10^6 times slower than memory access	Improving but gap remains significant	NVMe, storage class memory, computational storage	★★★★
Instruction-Level Parallelism	Diminishing returns beyond 4-6 issue width	Limits reached in most architectures	Branch prediction, speculative execution	★★★
Cache Coherence	Up to 40% overhead in heavily shared workloads	Worsens with core count	Directory-based protocols, relaxed consistency	★★★

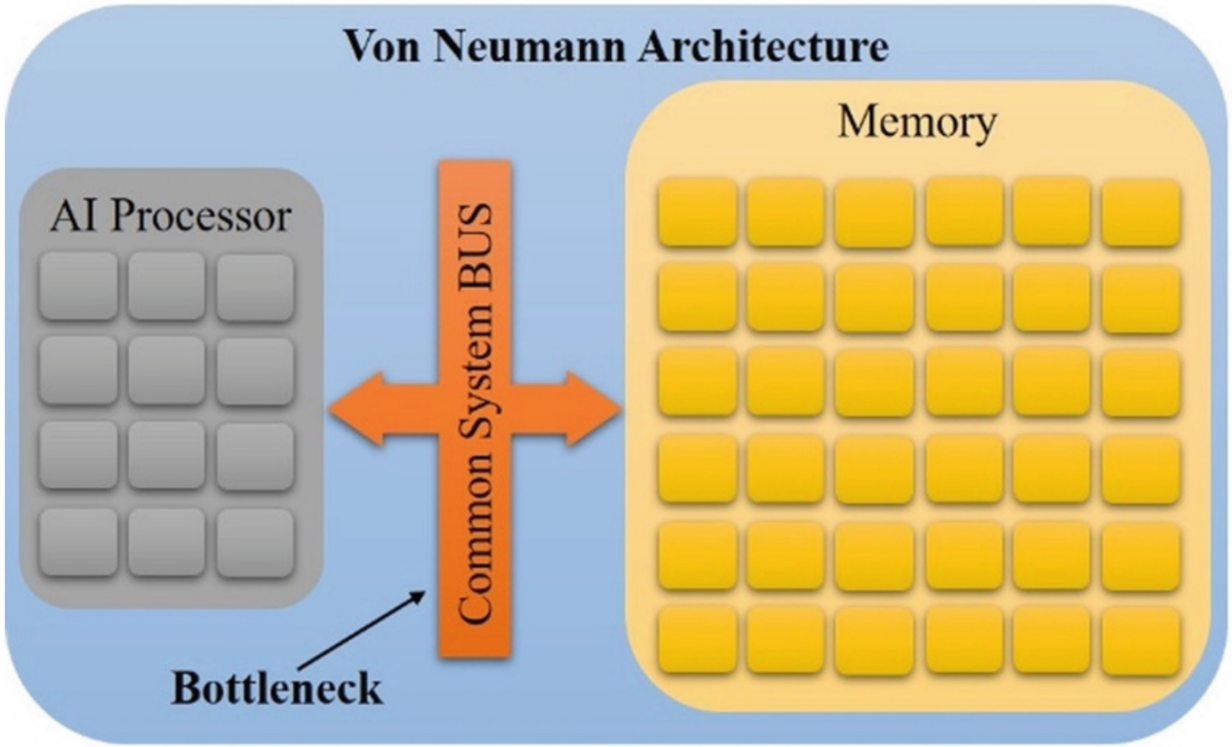
Innovations to address bottlenecks

Architecture Innovation	Memory Wall	Power	Sequential	I/O	ILP	Cache Coherence
Processing-in-Memory	✓✓✓	✓✓	✓	✓✓	-	✓
Domain-Specific Architectures	✓	✓✓✓	✓✓	-	✓✓	✓
Chiplet Designs	✓✓	✓✓	-	✓	-	-
Quantum Computing	✓	-	✓✓✓	-	✓✓✓	-
Neuromorphic Computing	✓✓	✓✓✓	✓✓	-	✓	✓✓

Checkmarks indicate level of impact in addressing the bottleneck (more checkmarks = greater impact)

Christof Teuscher • teuscher@pdx.edu

Non-von Neuman architectures
In-memory computation (IMC or PiM)



?

Fig. 27 Von Neumann versus non-Von Neumann architecture

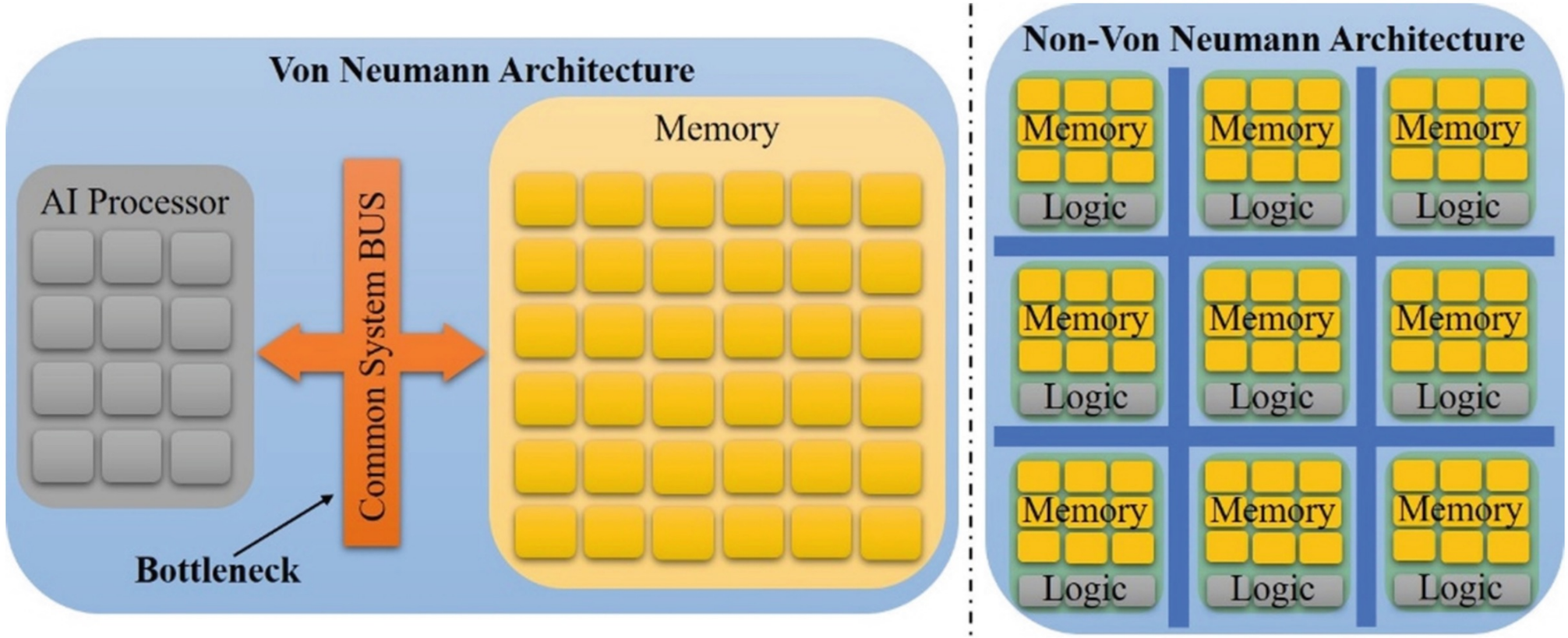


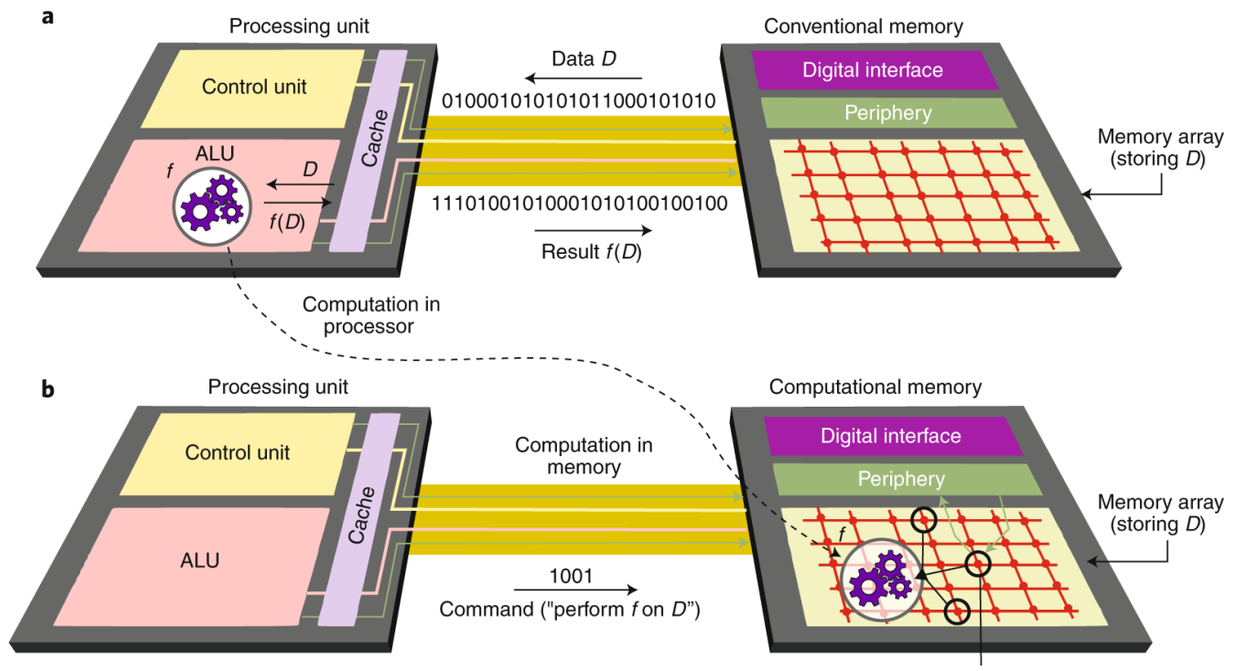
Fig. 27 Von Neumann versus non-Von Neumann architecture

QUIZ

Fundamental problem: separation of memory and computation. Communication cost is fundamental, computer architectures cannot avoid it, but they can attempt to amortize it.

Mishra et al.

IMC: the basic idea is simple



Sebastian et al., Nature

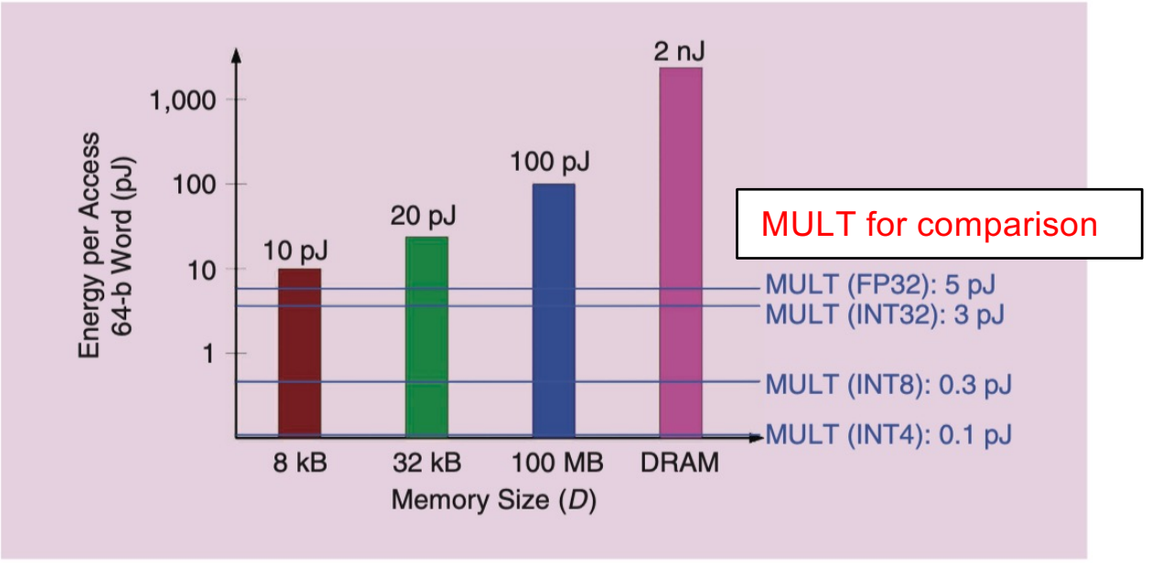


FIGURE 1: The energy cost of accessing memory.

A comparison of the energy required to access one word of data (64 b) from different-sized memories in a 45-nm technology to the energy of multiplication operations (considering the lowered precision levels that are increasingly relevant for deep-learning inference computations.

Verma et al., In-Memory Computing, 2019

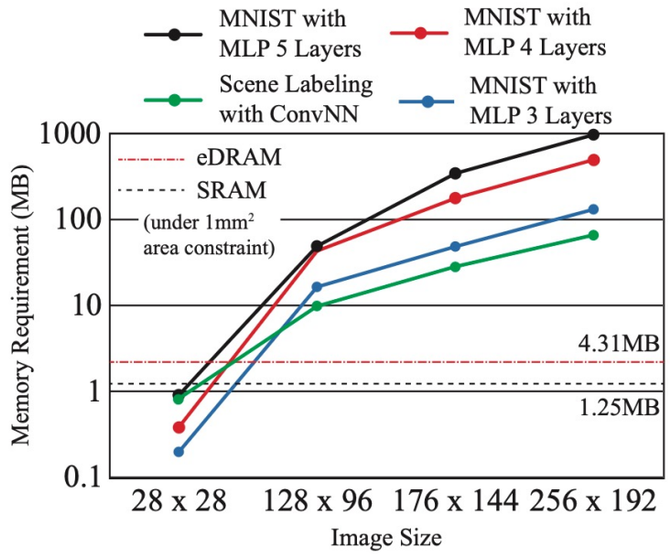
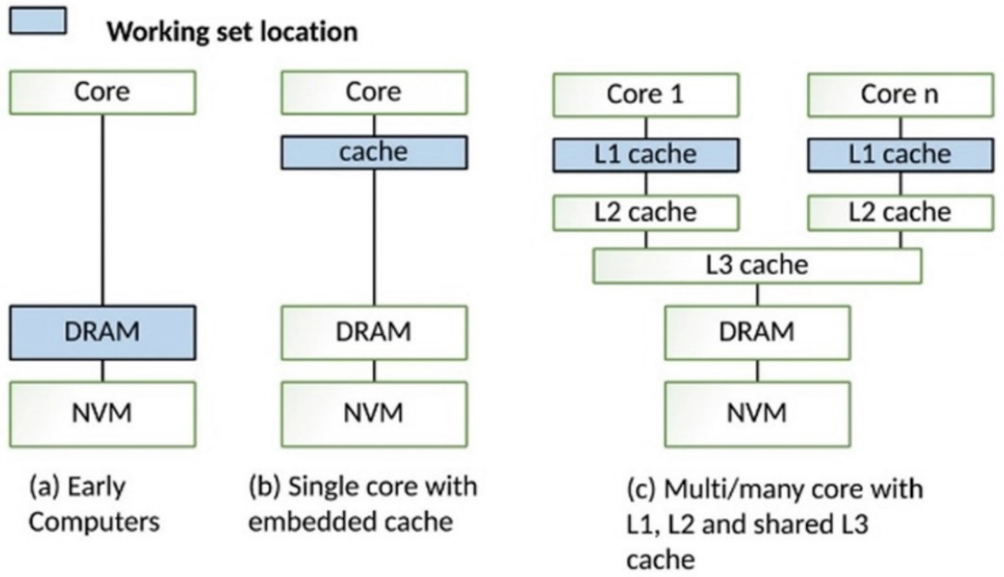
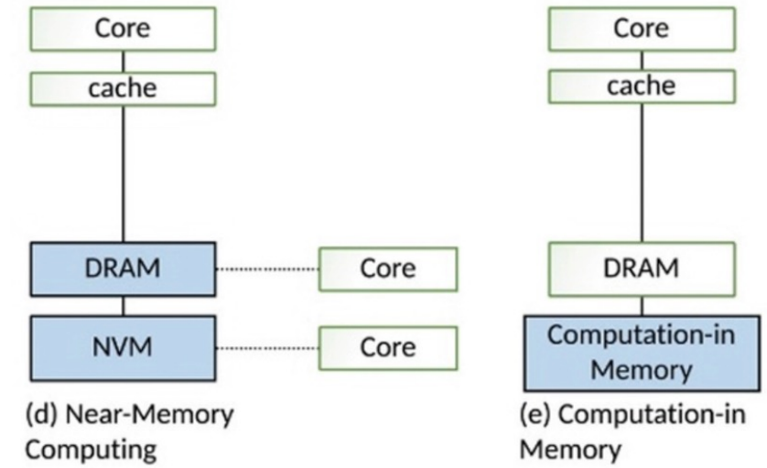
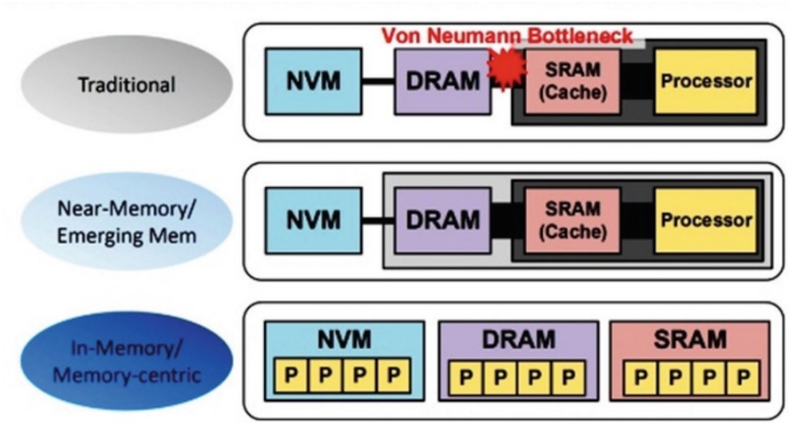


Figure 1. Required memory for scene labeling [9] with different image size using convolutional neural network and for MNIST [10]. Memory capacity of SRAM and eDRAM is normalized by $1mm^2$ area constraint [11], [12].

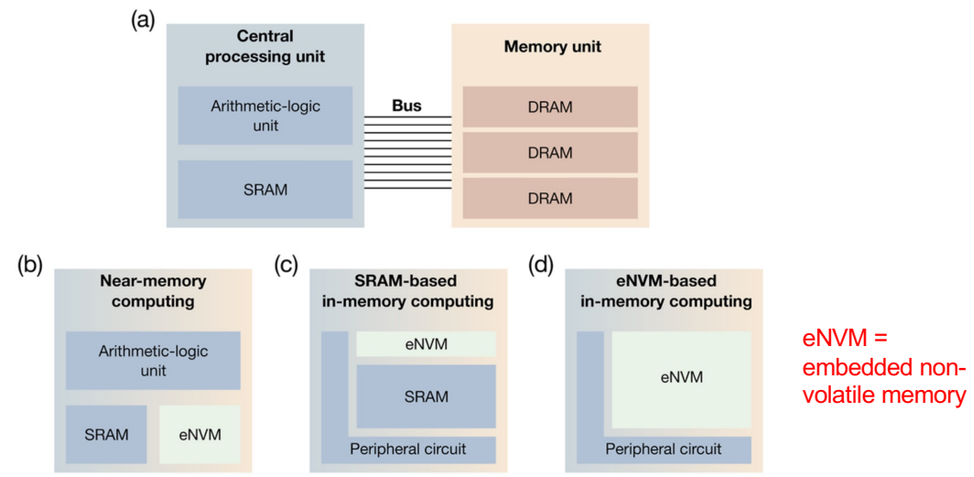
Kim et al., Neurocube







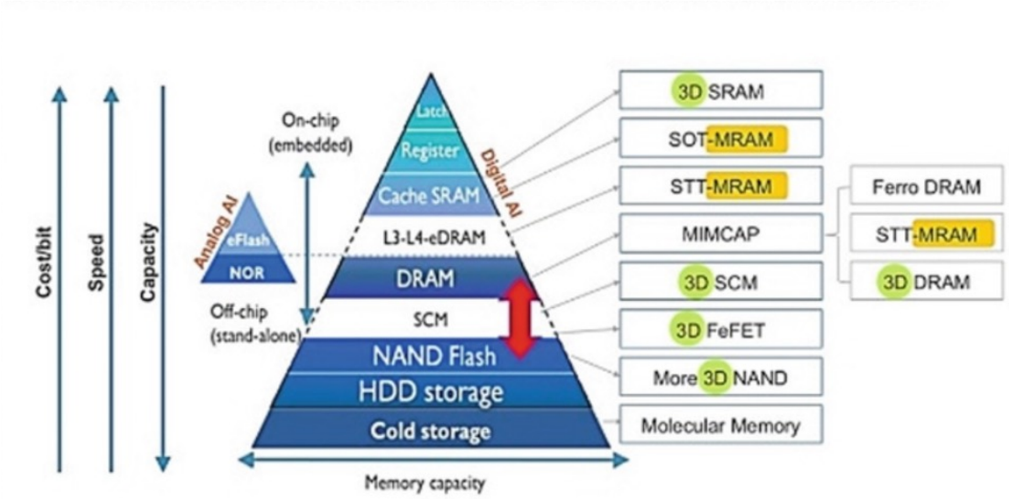
Mishra et al.



Mannocci et al.

eNVM = embedded non-volatile memory

It's all about trade-offs



Mishra et al.

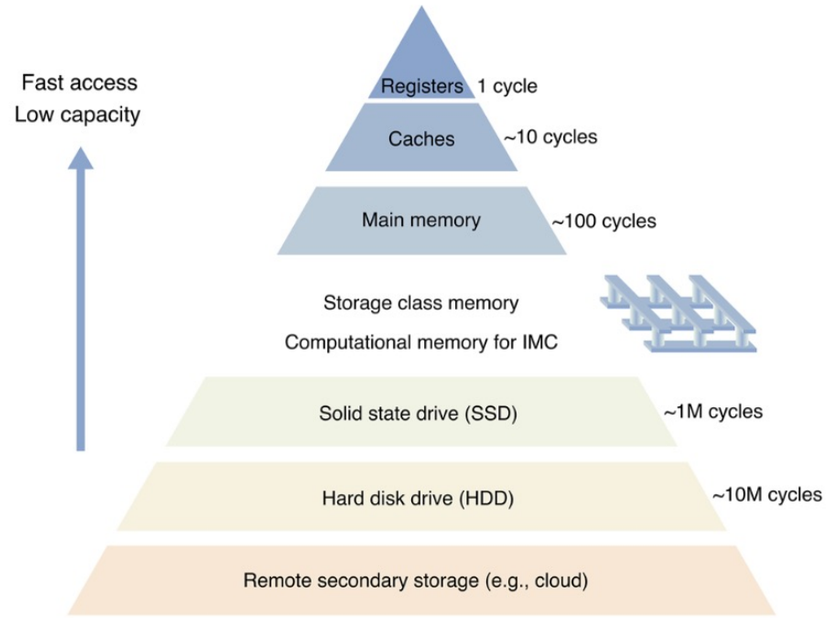


FIG. 2. Schematic illustration of the memory hierarchy in traditional CMOS-based computing systems. Registers and cache memories have relatively fast access and low capacity. Moving away from the CPU (top), memories increasingly display slower access and larger capacity. The storage class memory can bridge the gap between high-performance working memory and low-cost storage devices.

Mannocci et al.

IMC macro-categories

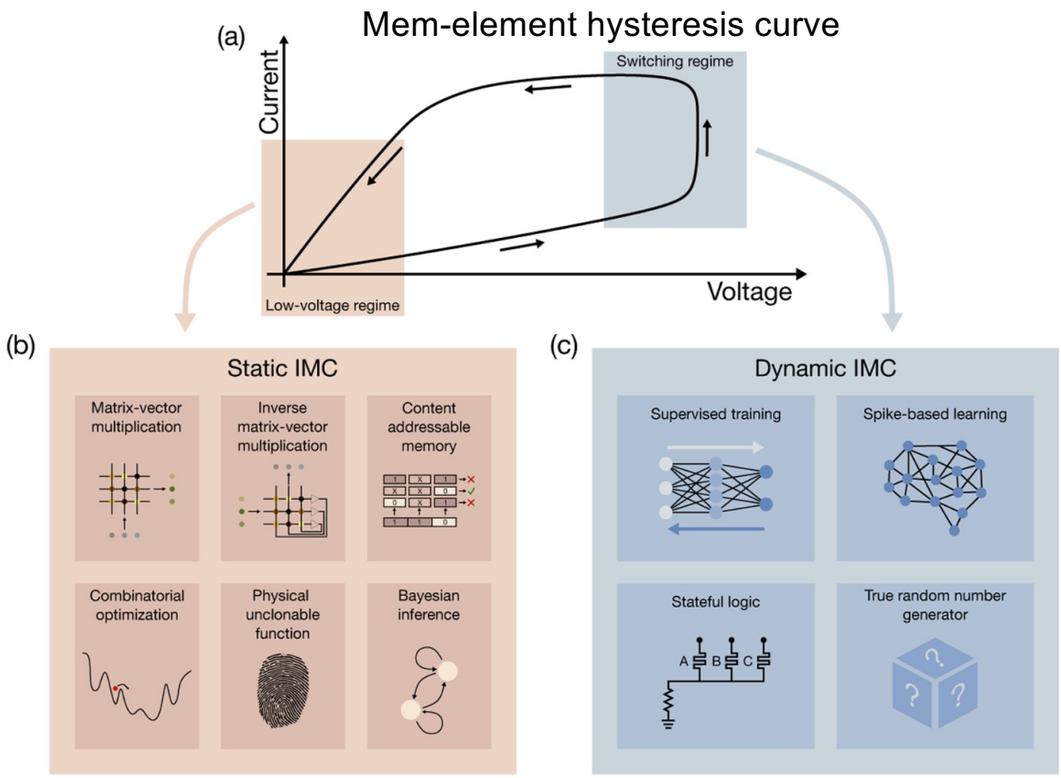
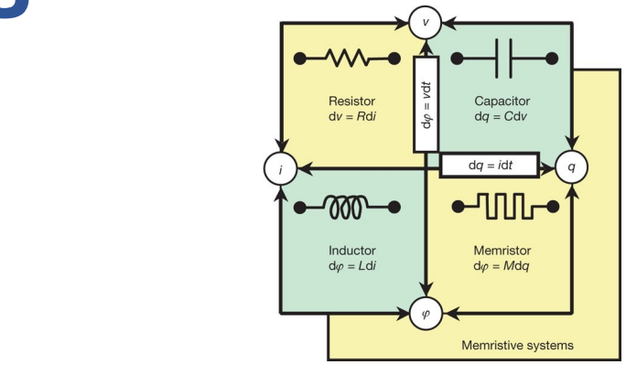
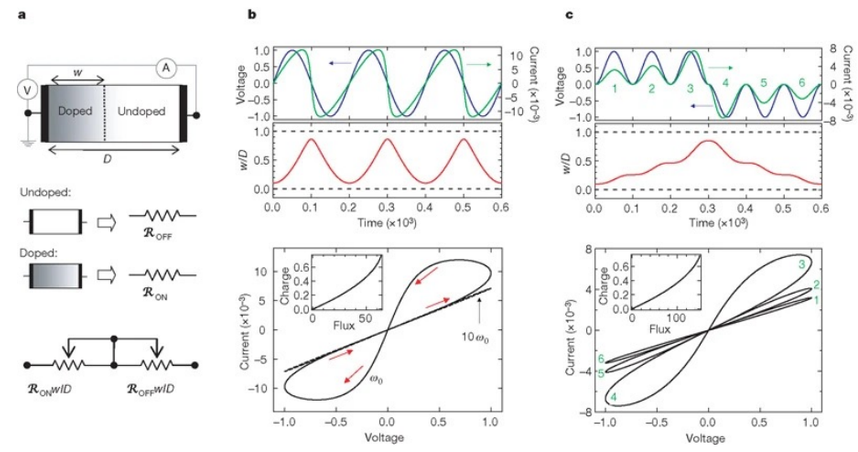


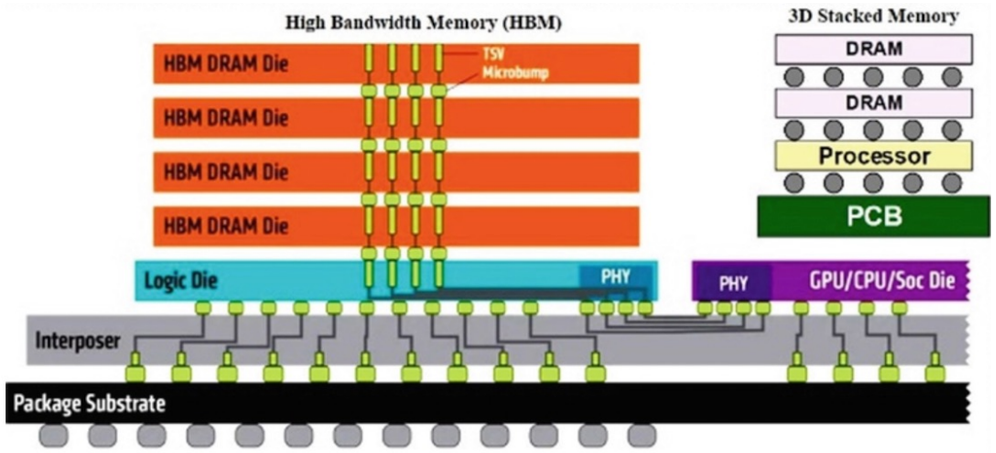
FIG. 4. IMC macro-categories and corresponding applications. (a) Schematic current–voltage (I–V) curve of an emerging memory device, highlighting the low-voltage and high-voltage/switching regimes, corresponding to static and dynamic IMC, respectively. (b) Examples of static IMC, where the memory stores pre-trained data and executes the computation, e.g., MVM. (c) Examples of dynamic IMC, in which the switching regime allows reproducing dynamic features, such as adaptation and learning.





Strukov, D., Snider, G., Stewart, D. *et al.* The missing memristor found. *Nature* **453**, 80–83 (2008). https://doi.org/10.1038/nature06932

Stacked 3D chips



Mishra et al.

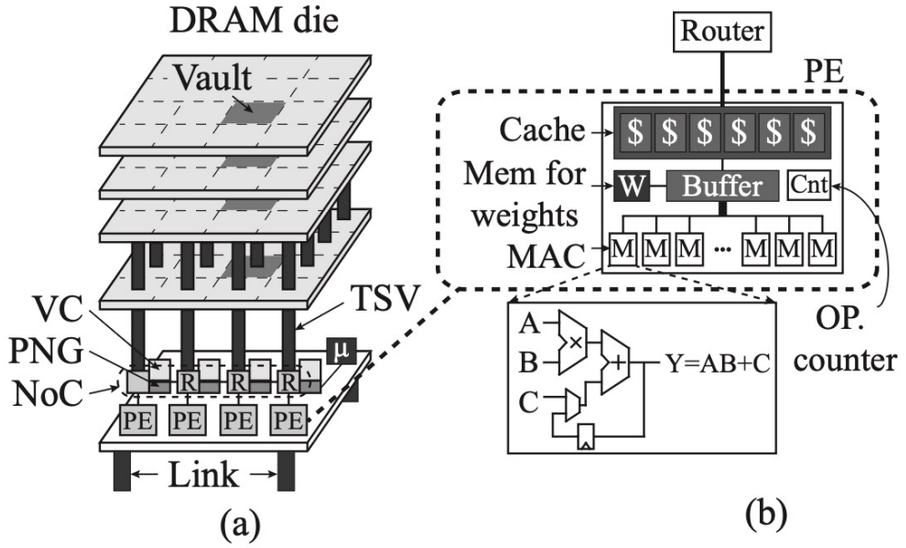
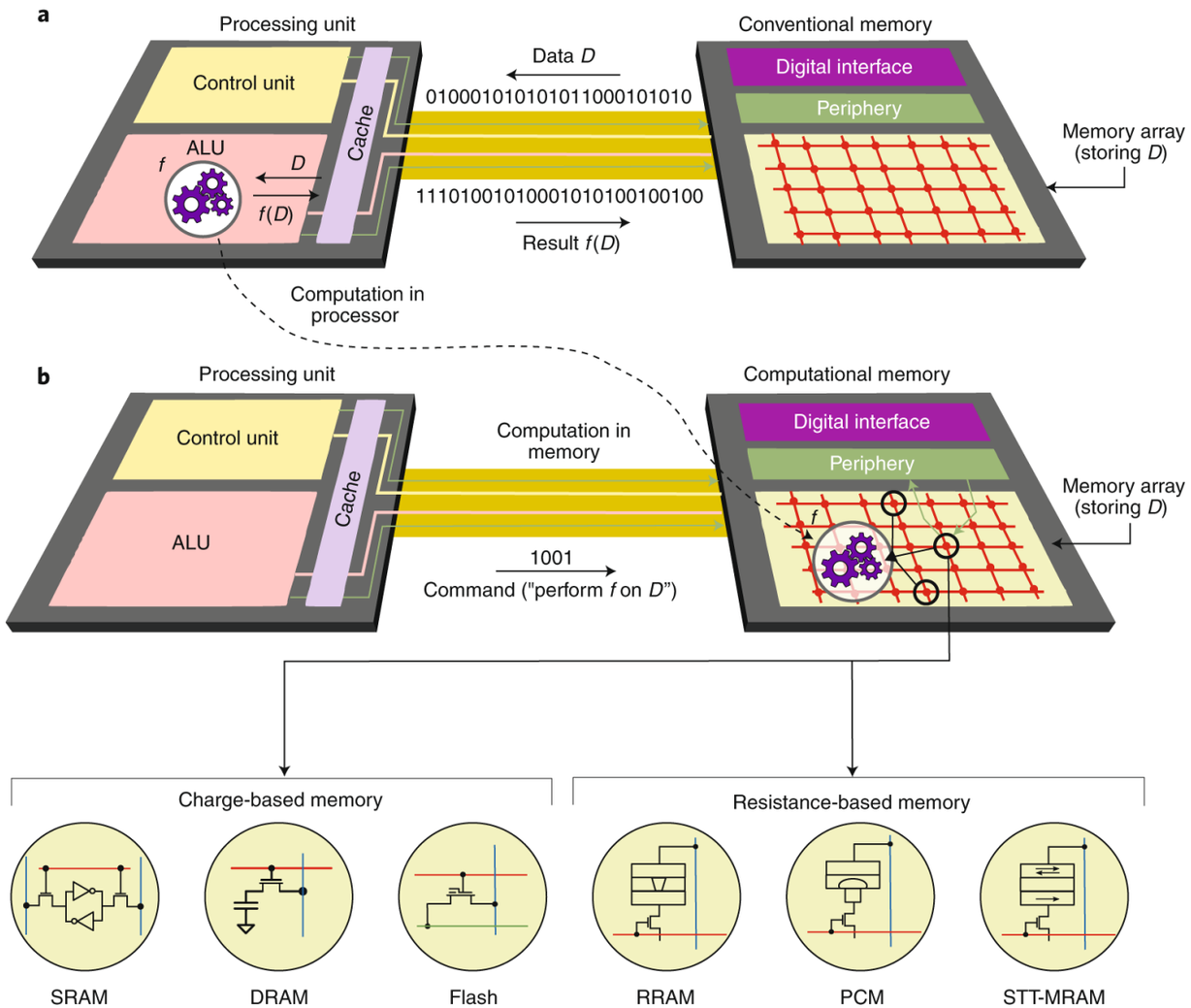


Figure 5. (a) Proposed Neurocube architecture and (b) Organization of the processing elements (PEs).

Kim et al., Neurocube



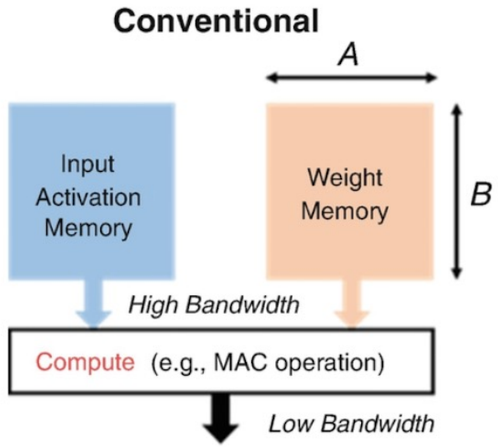
Property	RRAM	PCM	STT-MRAM
Mechanism	Filament formation	Phase change	Magnetic spin torque
Write speed	~10 ns	~50–100 ns	~2–10 ns
Endurance	10^6–10^{12}	10^6–10^8	>10^{12}
Analog (MLC)	Strong	Strong	Limited
Maturity	Research/ production	Research/ early prod.	Production (embedded)

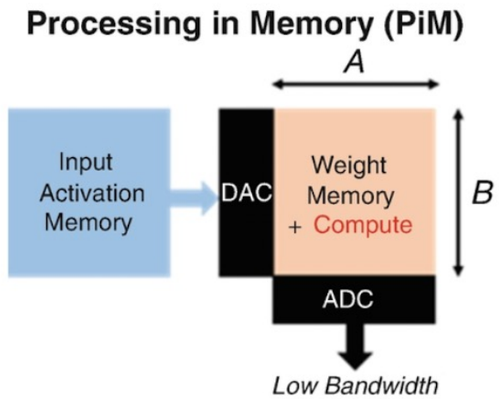
MLC = Multi-Level Cell (stores more than 1 bit)

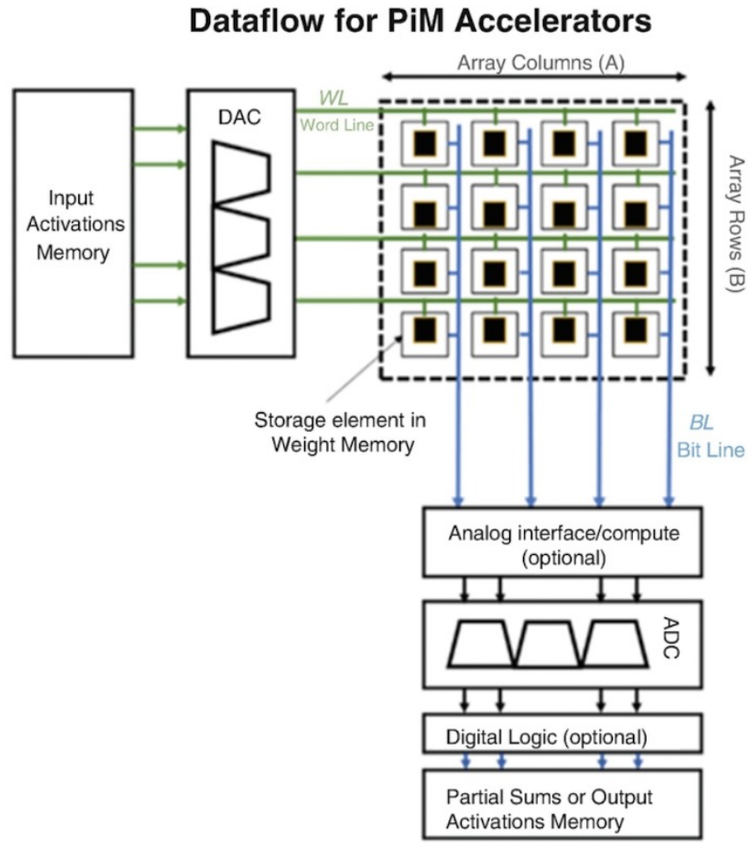
RRAM = Resistive RAM
PCM = Phase-Change Memory
STT-MRAM = Spin-Transfer Torque Magnetic RAM

Sebastian et al., Nature

Conventional vs Processing-in-Memory (PiM)







Mishra et al.

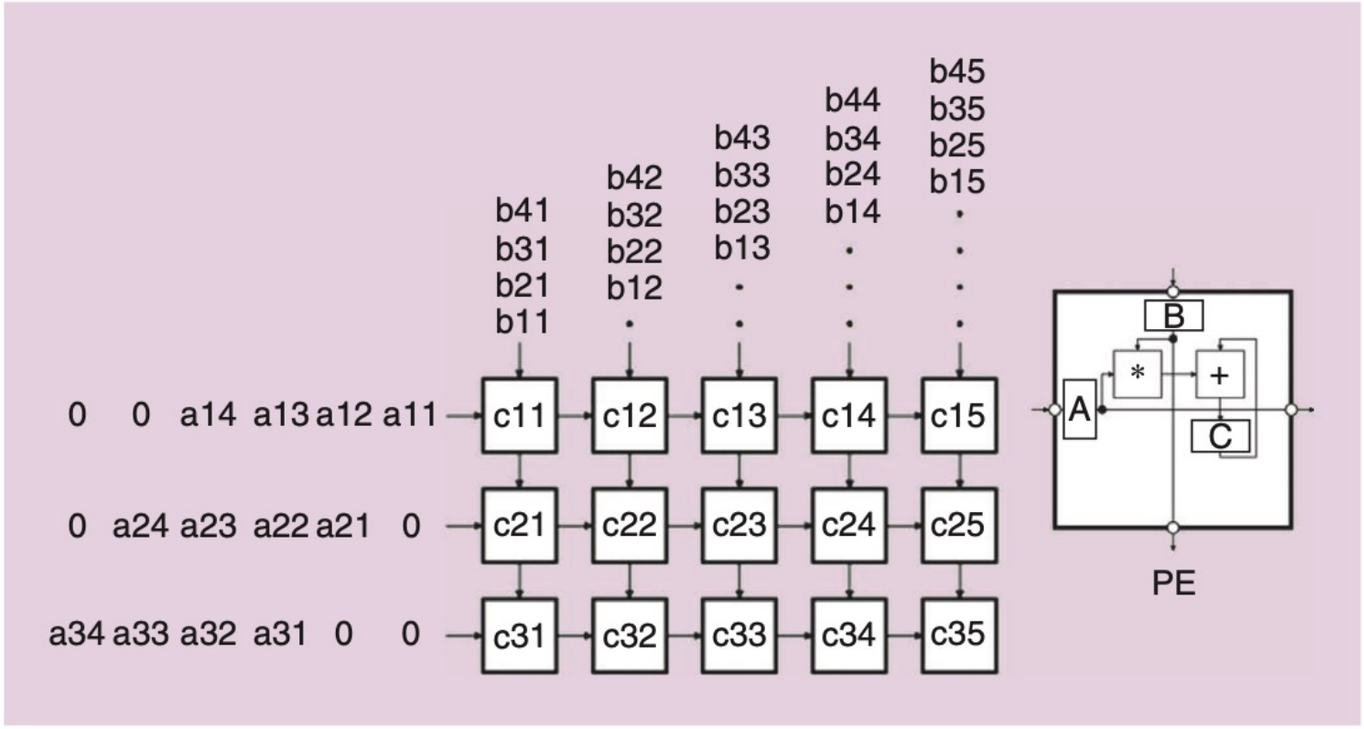


FIGURE 2: The spatial architecture for data-movement/memory-accessing amortization.

Matrix-vector multiplication (MCM)

$$\text{MVM } \vec{c} = \mathbf{A} \times \vec{b}$$

QUIZ

Verma et al., In-Memory Computing, 2019

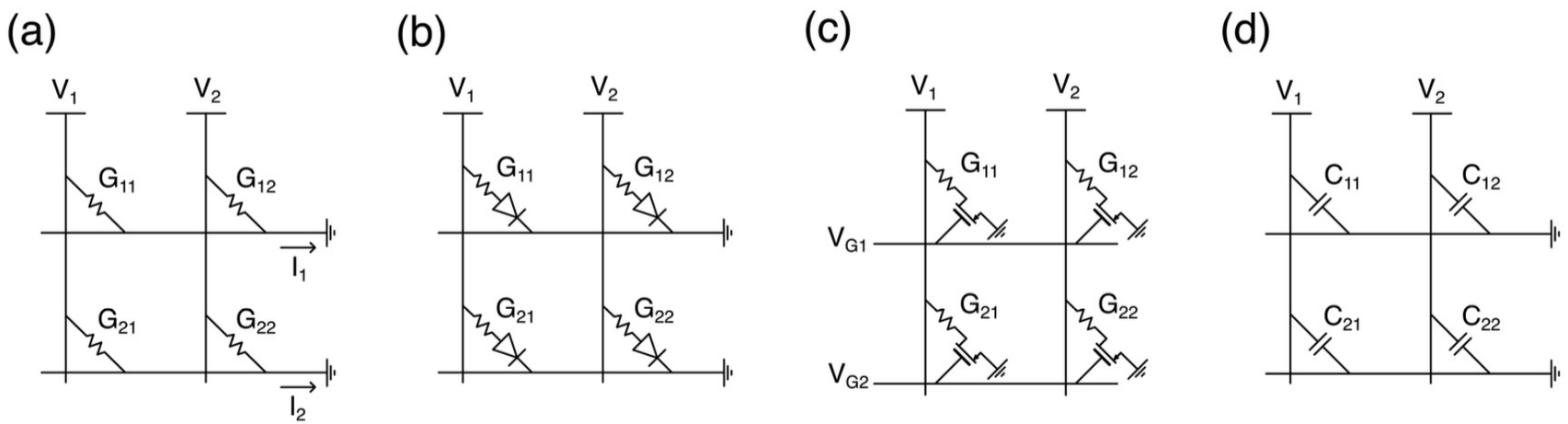
MVM (Matrix-Vector Multiplication) can be executed in a crosspoint memory array by universal circuit laws, such as Kirchhoff's current law for summation and Ohm's law for multiplication.

This is schematically shown in Fig. 5(a), where the application of a voltage V_j at the jth column results in a current at the ith row, connected to ground, given by

$$I_i = \sum_j^N G_{i,j} \cdot V_j, \quad (1)$$

where $G_{i,j}$ is the conductance of the memory element at position i,j and N is the number of rows and columns.[7,107] Equation (1) can be written in the compact matrix form $\mathbf{i} = \mathbf{Gv}$, thus evidencing the multiplication of the conductance matrix $\mathbf{G}$ with the voltage vector $\mathbf{v}$.

crosspoint array or **crossbar**

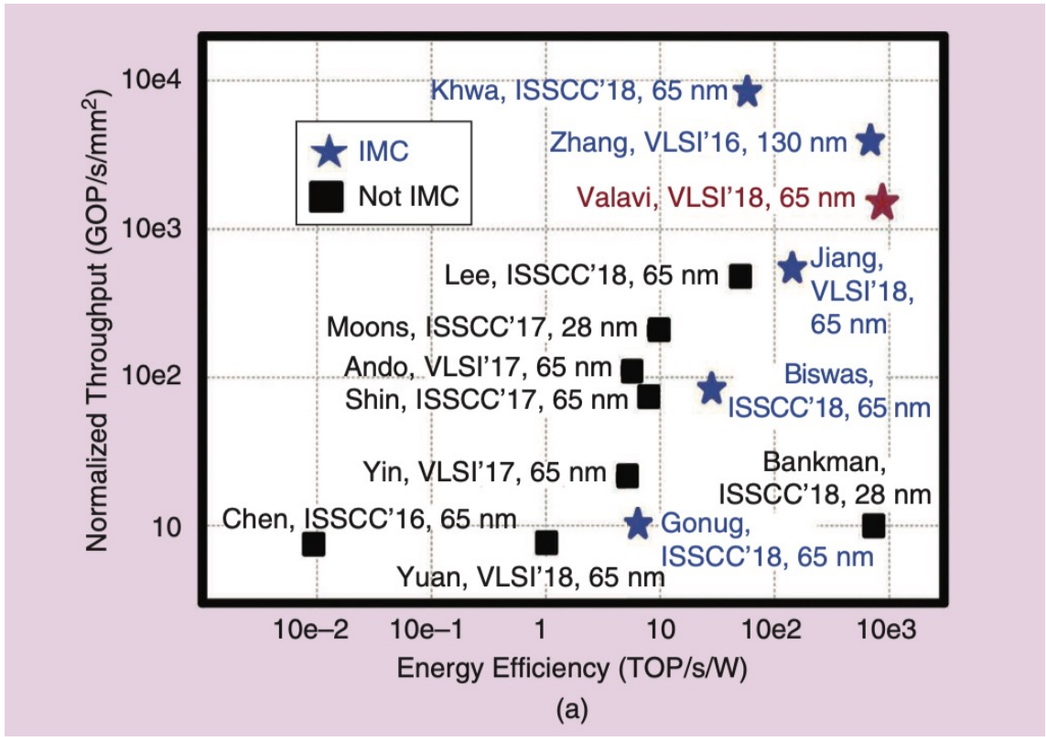


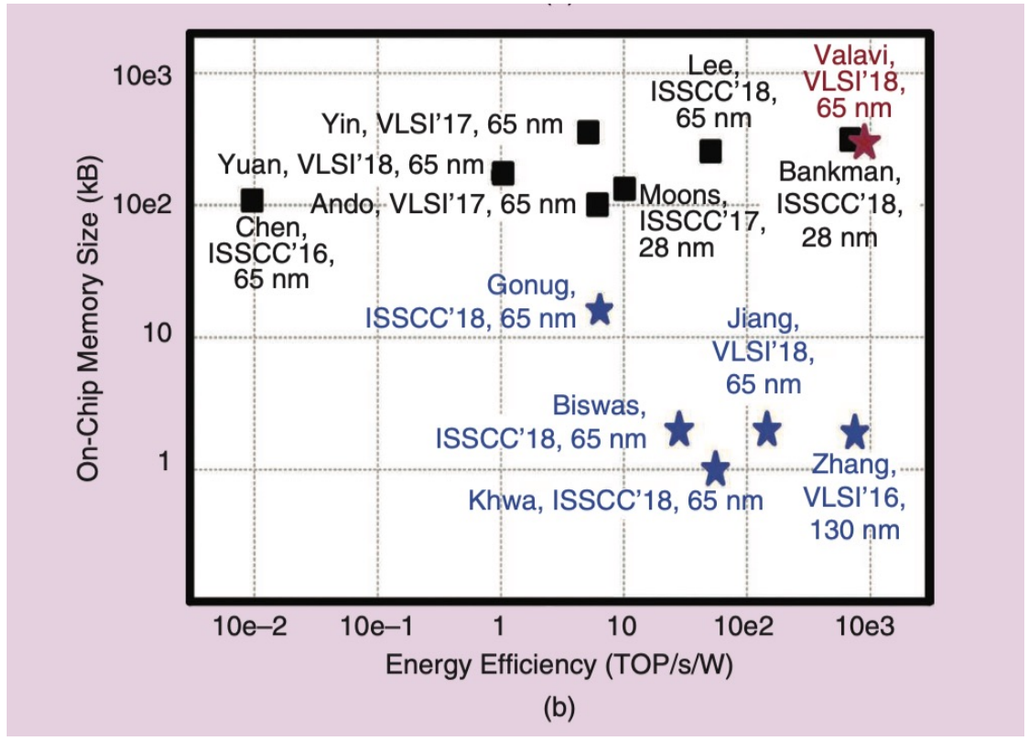
QUIZ

FIG. 5. Various cell structures for crosspoint array circuits. (a) One-resistor (1R) structure where the cell consists of a passive resistive device. (b) One-selector/one-resistor (1S1R) structure where the sneak path problem is circumvented by a non-linear selector device without affecting the integration density. (c) One-transistor/one-resistor (1T1R) structure allows for the selection of individual cells during programming and reading at the cost of a lower integration density. (d) One-capacitor (1C) structure, which prevents static leakage during MVM.

Mannocci et al.

IMC vs non-IMC comparison

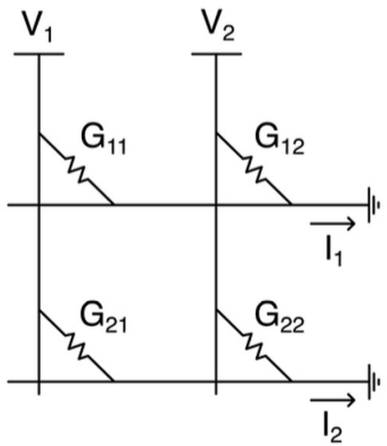




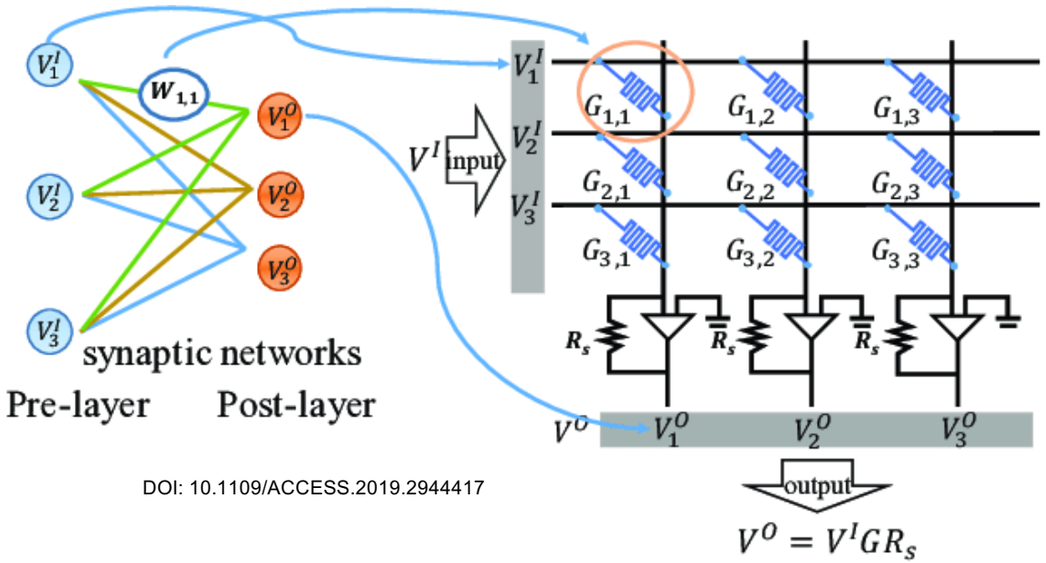
IMC enables ~ 10x gains in each metric.

Verma et al., In-Memory Computing, 2019

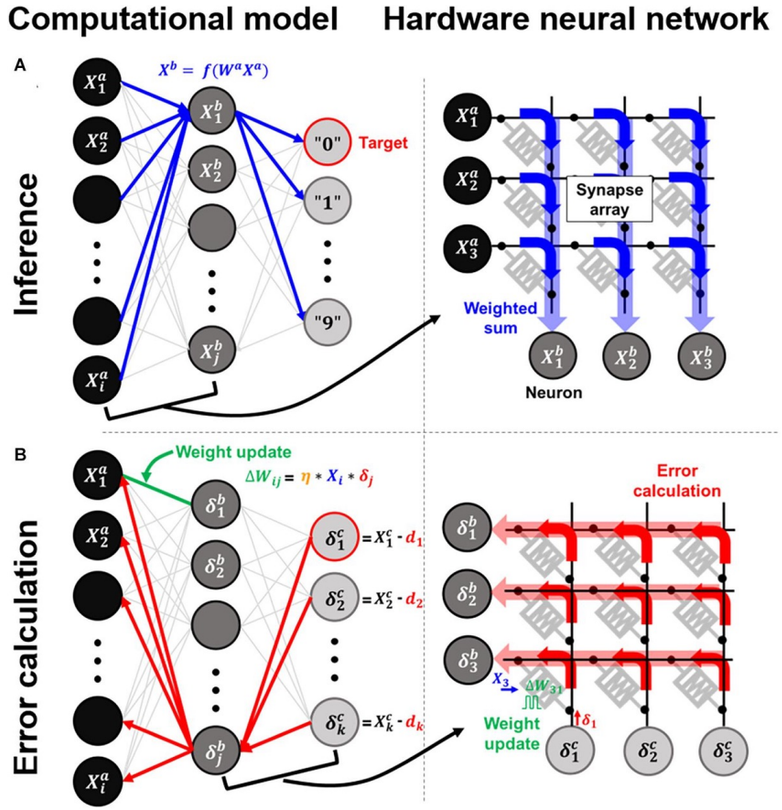
How can we map a NN onto a crossbar?



How can we map a NN onto a crossbar?



DOI: 10.1109/ACCESS.2019.2944417

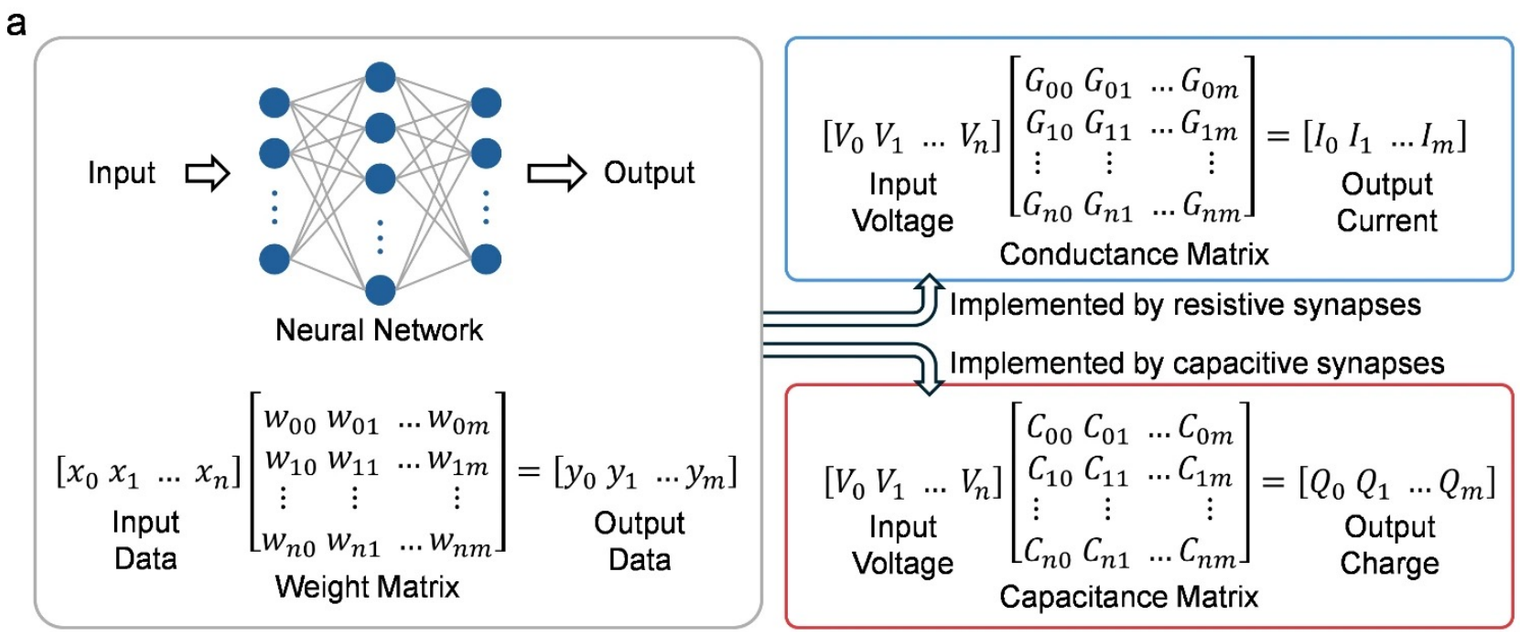


https://doi.org/10.3389/fnins.2021.690418

Parallel multiplication: When voltage is applied to the rows, the current flowing through each memristive junction is proportional to the product of the input voltage and the junction's conductance. This effectively performs multiplication at each intersection point simultaneously.
Current summation: The currents from all intersections in a column are naturally summed according to Kirchhoff's current law, effectively adding up all the partial products.

QUIZ

How can we map a NN onto a crossbar?

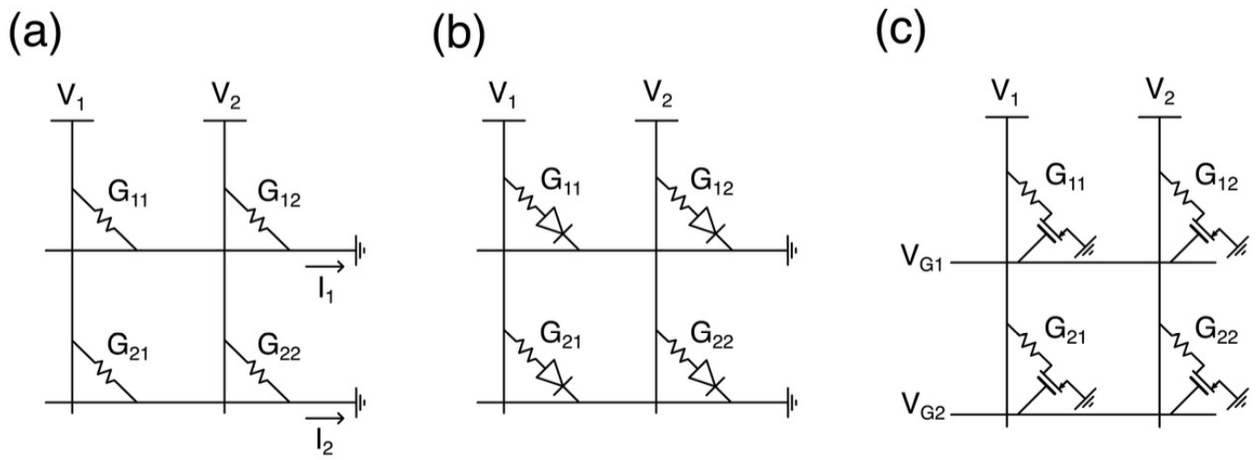


https://nanoconvergencejournal.springeropen.com/articles/10.1186/s40580-024-00463-0

Memristive vs memcapacitive crossbar

Property	Memristor crossbar	Memcapacitor crossbar
Weight encoding	Conductance G (resistance history)	Capacitance C (charge history)
Read operation	Apply V → measure I = V·G	Apply V → measure Q = V·C
Sneak path problem	Yes — resistive paths leak DC current	Greatly reduced — capacitors block DC
Spike compatibility	Optional	Natural — charge integration suits pulses
Non-volatile?	Yes	Yes (memcapacitive) but hard to fabricate
Maturity	Well demonstrated	Mostly theoretical / early research

Sneak paths



QUIZ

Sneak-paths in crossbars are unintended current pathways that occur in memory array architectures.

Solutions: **diodes** or **1T1R structures**

Questions

- What is the primary advantage of in-memory computing.
- In a typical crossbar implementation of a neural network, where are the synaptic weights physically represented?
- What are the benefits of spiking neural nets over analog neural nets?